

Implementation of Phase Field Method of Fracture using MOOSE

Submitted in partial fulfillment of the requirements of
the degree of Master of Technology

by

Ayush Saxena

Roll no: 22m1670

Supervisor: Prof. Tanmay K. Bhandakkar



DEPARTMENT OF MECHANICAL ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY BOMBAY

2023-24

Approval

This thesis/dissertation/report entitled **Implementation of Phase Field Method of Fracture using MOOSE** by Ayush Saxena, Roll No 22M1670 is approved for the degree of Master of Technology in Mechanical Engineering(Design).

Supervisor(s)

Examiners

Professor Tanmay K. Bhandakkar



Professor Salil S. Kulkarni

S.S. Kulkarni .

Chairman

S.S. Kulkarni

Professor Anirban Patra



Date: 26th June 2024

Place: Mechanical Department, IIT Bombay

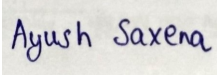
Declaration

I, Ayush Saxena, Roll No. 22M1670 understand plagiarism by carefully reading guidelines given at <https://ori.hhs.gov/avoiding-plagiarism-self-plagiarism-and-other-questionable-writing-practices-guide-ethical-writing>

I have made sure that all the ideas, expressions, graphs, diagrams, etc., that are not a result of my work, are properly credited. Long phrases or sentences that had to be used verbatim from published literature have been clearly identified using quotation marks.

I affirm that no portion of my work can be considered as plagiarism and I take full responsibility if such a complaint occurs. I understand fully well that the guide of the seminar report may not be in a position to check for the possibility of such incidences of plagiarism in this body of work.

I further affirm that I have not used any AI products, resources, or software such as ChatGPT for either writing or enhancing the text of this report.

Signature: 

Name: Ayush Saxena

Roll No.: 22M1670

Date: 20/06/2024

Acknowledgement

I would like to express my sincere gratitude to Prof. Tanmay K. Bhandakkar, Department of Mechanical Engineering, IIT Bombay, for the invaluable guidance and support he gave throughout this project. His regular interactions were instrumental in identifying and correcting the misunderstandings that I had, and his feedback with insightful comments greatly enhanced my project work.

Contents

	Page
List of Figures	vii
Abstract	viii
1 Phase-field Fracture Model	1
1.1 Introduction	1
1.2 The Griffith Energy Balance	1
1.3 Modeling brittle fracture using variational approach	1
1.4 Addition of Phase-field parameter	3
1.4.1 Governing equations in strong form for Quasi-Static brittle fracture	3
1.4.2 Governing equations for dynamic fracture	4
1.5 Geometric crack function	5
1.5.1 Classical Geometric crack function	5
1.5.2 Generic Geometric crack function	6
1.5.3 Length scale Parameter	6
1.6 Stored energy functional splitting models	7
1.6.1 Isotropic model with no split [5]	7
1.6.2 Anisotropic model of Lancioni and Royer-Carfagni [11]	7
1.6.3 Anisotropic model of Amor et al. [2]	8
1.6.4 Anisotropic model of Miehe et al. [15]	8
1.6.5 Anisotropic model of Wu and Nguyen [20]	8
1.7 Energetic degradation function	8
1.8 Governing equations in weak form	9
2 Using MOOSE for Solid Mechanics problems	10
2.1 Introduction	10
2.2 Uni-axial loading (Quasi-Static)	10
2.3 Thermo-mechanical problem	12
2.4 Uni-axial loading (Dynamic) [18]	13
3 Using MOOSE for benchmark Quasi-Static Phase-field problems	16
3.1 Introduction	16
3.2 Uni-axial tension applied to a plane strain domain [16]	16
3.2.1 Analytical Solution	17
3.2.2 Solution using phase field module	17
3.2.3 Solution using Staggered scheme	18
3.2.4 Comparison of results	27
3.3 Miehe Tension test [14]	27
3.3.1 Mesh and Discretization	28
3.3.2 Loading and Displacement	30
3.3.3 Solver and Tolerances	30
3.3.4 Results	30
3.3.5 Discussion	32
3.4 Al ₂ O ₃ and ZrO ₂ Tension test [16]	32
3.4.1 Mesh and Discretization	32
3.4.2 Loading and Displacement	33

3.4.3	Results	33
3.4.4	Discussion	34
3.5	Miehe Shear test [14]	35
3.5.1	Mesh and Discretization	35
3.5.2	Loading and Displacement	36
3.5.3	Results	37
3.5.4	Discussion	38
3.6	Symmetric 3-point bending test [14]	38
3.6.1	Mesh and Discretization	38
3.6.2	Loading and Displacement	39
3.6.3	Results	39
3.6.4	Discussion	40
3.7	Asymmetric notched beam test [14]	40
3.7.1	Mesh and Discretization	41
3.7.2	Loading and Displacement	41
3.7.3	Results	41
3.7.4	Discussion	42
3.8	L-shaped panel test [1]	43
3.8.1	Mesh and Discretization	43
3.8.2	Loading and Displacement	43
3.8.3	Results	44
3.8.4	Discussion	45
4	Conclusion	46
	Future work	46
	Appendix	47
	A1: Heat transfer Module	47
	A2: Application Development	52
	A3: Tensor Mechanics Module	53
	A4: Input files for Phase-field benchmark problems	55
	Bibliography	73

List of Figures

1.1	Representation of Phase-field parameter used to define a crack	3
1.2	Classical geometric crack function with different values of l_0	5
1.3	Generic geometric crack function	6
1.4	Variation of crack surface functional with l_0	6
2.1	Representation of displacement	10
2.2	variables change	11
2.3	Master action in Tensor Mechanics	11
2.4	Applying pressure in Boundary Condition block	11
2.5	Materials block	12
2.6	2D Plane Stress element undergoing thermal expansion	12
2.7	Changes in material block	12
2.8	AuxKernel block	13
2.9	Geometric and Boundary conditions for the dynamic place strain problem . .	13
2.10	Rate of traction applied	13
2.11	Displacement profile comparison	13
2.12	Comparing σ_{xx} profiles obtained from different sources	14
2.13	Adding Dynamic Master Action in MOOSE	14
2.14	Creating a linear piece-wise function	15
2.15	Executioner Block	15
3.1	Geometric and Boundary conditions for single element	16
3.2	moose solver	18
3.3	our solver	18
3.4	Picard iterations	19
3.5	Flow chart for staggered scheme	20
3.6	Terminal output for FixedPointPostProcessors	24
3.7	Final value of phase field parameter c	26
3.8	Final value of stress_yy	26
3.9	Phase-field parameter versus axial strain	27
3.10	Stress versus axial strain	27
3.11	Single edge-notched Tension specimen	28
3.12	Mesh iteration 1	28
3.13	Mesh iteration 2	28
3.14	Representation of crack by a single line	29
3.15	Representation of crack in 3 lines	30
3.16	Crack pattern at different times	31
3.17	Load deflection curves for Miehe tension test	31
3.18	Geometry and Boundary conditions for Al_2O_3 and ZrO_2 Tension test	32
3.19	Understanding crack initialization	33
3.20	Comparison of crack growth for different definitions of initial crack	33
3.21	Comparison of mesh near crack tip for different definitions of initial crack . .	34
3.22	Load-deflection curve for Al_2O_3 and ZrO_2 combined	34
3.23	Geometric and Boundary conditions for Miehe Shear Test	35
3.24	Mesh for representing crack through the phase-field parameter	36
3.25	Mesh for representing sharp crack	36

3.26	Single edge notched pure shear test crack pattern for different displacement load	37
3.27	Load-deflection curve for Single edge notched pure shear test	37
3.28	Geometry and Boundary conditions	38
3.29	Refined mesh near crack propagation zone	38
3.30	Representation of mesh near the notch with different approaches	39
3.31	Crack pattern for three-point bending test	39
3.32	Load-deflection curves for different meshes	40
3.33	Geometry and Boundary conditions for Asymmetric notched beam test . . .	40
3.34	Representation of mesh for asymmetric notched beam test	41
3.35	Final crack pattern for asymmetric notched beam test	41
3.36	Standard results i) Miehe et al. [14] ii) Miehe and Gueres [13] iii) Bittencourt et al. [4]	42
3.37	Load-displacement curves for the asymmetrically notched beam with standard result taken from Ambati et al. [1]	42
3.38	Geometry and Boundary conditions for L-shaped panel test	43
3.39	Mesh refinement for L-shaped panel	43
3.40	Loading rate on L-shaped panel	43
3.41	Crack growth at displacements i) 0.22 mm ii) 0.3 mm iii) 0.45 mm and iv) 1 mm	44
3.42	Final crack growth for two different refinement zones	44
3.43	Load vs deflection curve for L-shaped panel test	45
4.1	Representation of temperature profile taken along $y=0.5$	48
4.2	Mesh block of the input file	48
4.3	Variable block	48
4.4	Kernel block	49
4.5	Material block	49
4.6	BCs block	49
4.7	Executioner block	49
4.8	Output block	50
4.9	Representation of temperature profile of center	50
4.10	Mesh block of the input file	50
4.11	Time Derivative addition in kernels	51
4.12	Vector Postprocessor block	51
4.13	Representation of temperature profile of center	51
4.14	Changes in material block	52
4.15	Representation of temperature profile	52
4.16	Representation of temperature profile along $y=0.5$	53
4.17	Change in input file for using own kernel	53
4.18	Temptry1.C file in src-kernels	53
4.19	Representation of displacement profile along x-axis	54
4.20	Representation of S11	54
4.21	Representation of S11	55

Abstract

Fracture remains a prevalent failure mode in engineering materials and structures, making the prediction and prevention of crack-induced failure a paramount concern for design engineers. Recent advances in computational capabilities have opened new avenues for predicting material failure and enhancing our understanding of the fracture process. Traditional sharp crack discontinuity models face challenges when dealing with complex crack topologies. However, the emergence of phase-field modeling offers a diffuse, versatile solution. Phase field methods, are essentially variational approaches to fracture, extending Griffith's theory for brittle fracture. The resulting governing equations, derived from variational principles, provide a robust foundation for finite element implementation. Phase field models excel at handling crack initiation, propagation, and branching.

This work provides a comprehensive overview of the phase field method (PFM) model for quasi-static fracture in brittle materials. The first chapter is dedicated to the theoretical understanding of all the terminologies that are used while addressing phase-field problems and also contains important proofs. The second chapter builds toward understanding the MOOSE framework with the help of some solid mechanics problems where we learn how to define the problem with a set of codes in MOOSE and interpret its output. In the third chapter, we have shown how phase field problems are dealt with, analytically and for simulation using the Staggered scheme using a uni-axial plane strain problem. After that other benchmark problems are worked out and through them, we get to understand what is the role of the different parameters involved in the phase field literature. Especially the non-linear zone where any change in parameter can lead to a significant change in material response and crack growth.

Chapter 1

Phase-field Fracture Model

1.1 Introduction

Fracture is a prevalent failure mode in engineering materials and structures, demanding proactive solutions. Computational modeling of fracture plays a crucial role, aiding predictions and offering insights into the behavior of various materials, from concrete to biological tissues. The historical foundations of Linear Elastic Fracture Mechanics (LEFM) provide valuable insights into crack initiation and propagation. Yet, the complexity of real-world fracture phenomena remains a challenge. For complex fracture problems involving intersecting cracks, numerical modeling remains a challenge, particularly in three dimensions.

To address these issues, automatic determination of crack paths is introduced in phase-field fracture/damage models (PFM). These models minimize the total potential energy of cracking solids, offering solutions without the need for derivatives. PFMs can handle complex fractures, including intersecting and branching cracks. This report provides an overview of computational fracture mechanics, highlighting the role of PFMs. Earlier we had just the equilibrium equations and Griffith's law for dealing with fracture problems, the introduction of the phase-field parameter introduces a phase-field evolution law which also affects stress. We have two equations now which are coupled partial differential equations and hence their analytical solution becomes tough to obtain. Numerically also, they are very computationally expensive. That's why we need dedicated software that can provide us with the solution to those equations. But even with software, we need to provide it with the complete problem statement. This requires an in-depth understanding of how we arrived at those equations. Hence, let's start understanding.

1.2 The Griffith Energy Balance

"A crack can form (or an existing crack can grow) only if such a process causes the total energy to decrease or remain constant. Thus the critical conditions for fracture can be defined as the point where crack growth occurs under equilibrium conditions, with no net change in total energy." [3]

Surface Energy increased + Potential Energy decreased = 0

$$\frac{dE}{dA} = \frac{d\Pi}{dA} + \frac{dW_s}{dA} \quad (1.1)$$

or

$$-\frac{d\Pi}{dA} = \frac{dW_s}{dA} \quad (1.2)$$

where, E = Total Energy, Π = Potential Energy supplied by the internal strain energy and external forces and W_s = Work required to create new surfaces

1.3 Modeling brittle fracture using variational approach

The total energy functional $\mathcal{E}$ in a quasi-static loading regime as defined in [21] is

$$\mathcal{E} := \Psi_s + \Psi_c - \mathcal{P} \quad (1.3)$$

where, Ψ_s = The stored strain energy (volume integral), Ψ_c = The surface energy (surface integral), and $\mathcal{P}$ = The external potential energy

$$\mathcal{E} = \int_{\Omega \setminus \Gamma} \psi_0(\boldsymbol{\epsilon}(\mathbf{u}), \Gamma) dV + \int_{\Gamma} G_c dA - \int_{\Omega} \mathbf{b}^* \cdot \mathbf{u} dV - \int_{\partial\Omega_t} \mathbf{t}^* \cdot \mathbf{u} dA \quad (1.4)$$

The initial (elastic) strain energy density function is dependent upon the strain tensor and the crack set Γ and G_c is a material property. We have a variational problem with the following conditions,

$$\dot{\Gamma}(t) \geq 0 \quad (1.5)$$

This is the irreversibility condition of the crack. It means a crack should always grow.

$$\delta \mathcal{E}(\mathbf{u}, \Gamma) \geq 0 \quad (1.6)$$

The pair $(\mathbf{u}, \Gamma)$ is a stationary point of the energy functional, so the above condition is always followed for any admissible variations of δu satisfying $\delta u = 0$ on the boundary $\partial\Omega_u$ and $\delta\Gamma \geq 0$

$$\begin{aligned} \delta \mathcal{E} &= \frac{\partial \Psi_s}{\partial \boldsymbol{\epsilon}_{ij}} \delta \boldsymbol{\epsilon}_{ij} + \frac{\partial \Psi_s}{\partial \Gamma} \delta \Gamma + \frac{\partial \Psi_c}{\partial \Gamma} \delta \Gamma - \int_{\Omega} \mathbf{b}^* \cdot \delta \mathbf{u} dV - \int_{\partial\Omega_t} \mathbf{t}^* \cdot \delta \mathbf{u} dA \\ \delta \mathcal{E} &= \underbrace{\int_{\Omega \setminus \Gamma} \frac{\partial \psi_0}{\partial \boldsymbol{\epsilon}_{ij}} \delta \boldsymbol{\epsilon}_{ij} dV}_{\text{Part I}} + \left(\frac{\partial \Psi_s}{\partial \Gamma} + G_c \right) \delta \Gamma - \int_{\Omega} \mathbf{b}^* \cdot \delta \mathbf{u} dV - \int_{\partial\Omega_t} \mathbf{t}^* \cdot \delta \mathbf{u} dA \\ \text{Part I} &= \int_{\Omega \setminus \Gamma} \frac{1}{2} \boldsymbol{\sigma}_{ij} \delta \left(\frac{\partial \mathbf{u}_i}{\partial \mathbf{x}_j} + \frac{\partial \mathbf{u}_j}{\partial \mathbf{x}_i} \right) dV = \int_{\Omega \setminus \Gamma} \frac{1}{2} \boldsymbol{\sigma}_{ij} \left(\frac{\partial \delta \mathbf{u}_i}{\partial \mathbf{x}_j} + \frac{\partial \delta \mathbf{u}_j}{\partial \mathbf{x}_i} \right) dV \\ &= \int_{\Omega \setminus \Gamma} \frac{1}{2} (\boldsymbol{\sigma}_{ij} \delta \partial \mathbf{u}_{i,j} + \boldsymbol{\sigma}_{ij} \delta \partial \mathbf{u}_{j,i}) dV = \int_{\Omega \setminus \Gamma} \frac{1}{2} (\boldsymbol{\sigma}_{ij} \delta \partial \mathbf{u}_{i,j} + \boldsymbol{\sigma}_{ji} \delta \partial \mathbf{u}_{j,i}) dV \\ &= \int_{\Omega \setminus \Gamma} \boldsymbol{\sigma}_{ij} \delta \partial \mathbf{u}_{i,j} dV = \int_{\Omega \setminus \Gamma} \boldsymbol{\sigma}_{ij} \frac{\partial}{\partial \mathbf{x}_j} (\delta \mathbf{u}_i) dV = \int_{\Omega \setminus \Gamma} \frac{\partial}{\partial \mathbf{x}_j} (\boldsymbol{\sigma}_{ij} \delta \mathbf{u}_i) dV - \int_{\Omega \setminus \Gamma} \frac{\partial \boldsymbol{\sigma}_{ij}}{\partial \mathbf{x}_j} \delta \mathbf{u}_i dV \\ &= \int_{\partial\Omega} (\boldsymbol{\sigma}_{ij} \delta \mathbf{u}_i) \mathbf{n}_j dA - \int_{\Omega \setminus \Gamma} (\nabla \cdot \boldsymbol{\sigma}_{ij}) \delta \mathbf{u}_i dV \\ \delta \mathcal{E} &= \int_{\Gamma} (\boldsymbol{\sigma} \cdot \mathbf{n}) \cdot \delta \mathbf{u} dA + \int_{\partial\Omega_t} (\boldsymbol{\sigma} \cdot \mathbf{n} - \mathbf{t}^*) \cdot \delta \mathbf{u} dA - \int_{\Omega \setminus \Gamma} (\nabla \cdot \boldsymbol{\sigma} + \mathbf{b}^*) \cdot \delta \mathbf{u} dV \\ &\quad + \left(\frac{\partial \Psi_s}{\partial \Gamma} + G_c \right) \delta \Gamma \geq 0 \end{aligned} \quad (1.7)$$

This has to be true for any variation of δu and $\delta\Gamma > 0$ which gives us the following conditions

$$\boldsymbol{\sigma} \cdot \mathbf{n} = 0 \quad \text{on } \Gamma \quad (1.8)$$

$$\boldsymbol{\sigma} \cdot \mathbf{n} = \mathbf{t}^* \quad \text{on } \partial\Omega_t \quad (1.9)$$

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b}^* = 0 \quad \text{in } \Omega \setminus \Gamma \quad (1.10)$$

and,

$$\delta \mathcal{E} = (-\mathcal{G} + G_c) \delta \Gamma \geq 0, \quad \mathcal{G} - G_c \leq 0 \quad (1.11)$$

$\mathcal{G}$ is known as the energy release rate or crack driving force

The pair $(\mathbf{u}(t), \Gamma(t))$ is a global minimizer of the potential energy functional $\mathcal{E}$, i.e.,

$$(\mathbf{u}(t), \Gamma(t)) = \text{Arg}\{\min \mathcal{E}(\mathbf{u}, \Gamma)\} \quad (1.12)$$

1.4 Addition of Phase-field parameter

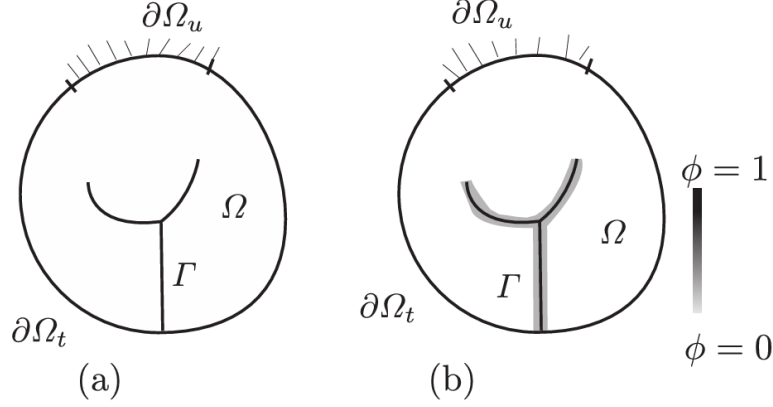


Figure 1.1: Representation of Phase-field parameter used to define a crack

So now we introduce the phase field parameter. The crack set Γ is now converted to a crack surface functional which is presented as,

$$\Gamma \approx \Gamma_l(\phi) = \int_{\mathcal{B}} \gamma(\phi; \nabla \phi) dV \quad (1.13)$$

where $\gamma(\phi; \nabla \phi)$ is known as the crack surface density function. So, now the surface energy can be written as,

$$\Psi_c := \int_{\Gamma} G_c dA = \int_{\mathcal{B}} G_c \gamma(\phi; \nabla \phi) dV \quad (1.14)$$

The elastic energy density function has also become now function of both $\epsilon(\mathbf{u})$ and ϕ

$$\Psi_s(\mathbf{u}, \phi) = \int_{\Omega} \psi(\epsilon(\mathbf{u}), \phi) dV \quad (1.15)$$

Accordingly, we have the stress tensor defined as,

$$\boldsymbol{\sigma} = \frac{\partial \psi}{\partial \epsilon} \quad (1.16)$$

This is the final total energy functional that we have now,

$$\mathcal{E} = \int_{\Omega} \psi(\epsilon(\mathbf{u}), \phi) dV + \int_{\mathcal{B}} G_c \gamma(\phi; \nabla \phi) dV - \int_{\Omega} \mathbf{b}^* \cdot \mathbf{u} dV - \int_{\partial \Omega_t} \mathbf{t}^* \cdot \mathbf{u} dA \quad (1.17)$$

1.4.1 Governing equations in strong form for Quasi-Static brittle fracture

We consider the following variational problem,

$$(\mathbf{u}(x), \phi(x)) = \text{Arg}\{\min \mathcal{E}(\mathbf{u}, \phi)\} \quad \text{subjected to} \quad \dot{\phi} \geq 0, \quad \phi \in [0, 1] \quad (1.18)$$

$$\begin{aligned} \delta \mathcal{E} = & \int_{\Omega} \boldsymbol{\sigma} \delta \epsilon dV + \int_{\mathcal{B}} \frac{\partial \psi}{\partial \phi} \delta \phi dV + \int_{\mathcal{B}} G_c \left(\frac{\partial \gamma}{\partial \phi} \delta \phi + \frac{\partial \gamma}{\partial \nabla \phi} \cdot \delta \nabla \phi \right) dV \\ & - \int_{\Omega} \mathbf{b}^* \cdot \delta \mathbf{u} dV - \int_{\partial \Omega_t} \mathbf{t}^* \cdot \delta \mathbf{u} dA \end{aligned} \quad (1.19)$$

$$\Rightarrow \frac{\partial \gamma}{\partial \nabla \phi} \cdot \delta \nabla \phi = \frac{\partial \gamma}{\partial \nabla \phi} \cdot \nabla (\delta \phi) = \nabla \left(\frac{\partial \gamma}{\partial \nabla \phi} \delta \phi \right) - \nabla \left(\frac{\partial \gamma}{\partial \nabla \phi} \right) \delta \phi$$

$$\begin{aligned}\delta\mathcal{E} = & - \int_{\Omega} (\nabla \boldsymbol{\sigma} + \mathbf{b}^*) \cdot \delta \mathbf{u} dV + \int_{\partial\Omega_t} (\boldsymbol{\sigma} \cdot \mathbf{n} - \mathbf{t}^*) \cdot \delta \mathbf{u} dA + \int_{\mathcal{B}} \frac{\partial\psi}{\partial\phi} \delta\phi dV \\ & + \int_{\mathcal{B}} G_c \left(\frac{\partial\gamma}{\partial\phi} - \nabla \left(\frac{\partial\gamma}{\partial\nabla\phi} \right) \right) \delta\phi dV + \int_{\mathcal{B}} G_c \nabla \left(\frac{\partial\gamma}{\partial\nabla\phi} \delta\phi \right) dV\end{aligned}\quad (1.20)$$

$$\begin{aligned}\delta\mathcal{E} = & - \int_{\Omega} (\nabla \boldsymbol{\sigma} + \mathbf{b}^*) \cdot \delta \mathbf{u} dV + \int_{\partial\Omega_t} (\boldsymbol{\sigma} \cdot \mathbf{n} - \mathbf{t}^*) \cdot \delta \mathbf{u} dA \\ & + \int_{\mathcal{B}} \left[\frac{\partial\psi}{\partial\phi} + G_c \delta\phi\gamma \right] \delta\phi dV + \int_{\partial\mathcal{B}} G_c \left(\frac{\partial\gamma}{\partial\nabla\phi} \cdot \mathbf{n}_{\mathcal{B}} \right) \delta\phi dA\end{aligned}\quad (1.21)$$

with $\delta\phi\gamma$ being the functional derivative

$$\delta\phi\gamma := \frac{\partial\gamma}{\partial\phi} - \nabla \left(\frac{\partial\gamma}{\partial\nabla\phi} \right) \quad (1.22)$$

and $\mathbf{n}_{\mathcal{B}}$ is the outward unit normal vector of the boundary $\partial\mathcal{B}$ of the localisation band $\mathcal{B}$.

The unilateral stationary condition of the total energy functional i.e., $\delta\mathcal{E} = 0$ for $\delta\phi > 0$ and $\delta\mathcal{E} > 0$ for $\delta\phi = 0$, gives us the following conditions

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b}^* = 0 \quad \text{in } \Omega \quad (1.23)$$

$$\boldsymbol{\sigma} \cdot \mathbf{n} = \mathbf{t}^* \quad \text{on } \partial\Omega_t \quad (1.24)$$

as well as

$$\begin{cases} Y - G_c \delta\phi\gamma = 0 & \delta\phi > 0 \\ Y - G_c \delta\phi\gamma < 0 & \delta\phi = 0 \end{cases} \quad \text{in } \mathcal{B} \quad (1.25)$$

$$\frac{\partial\gamma}{\partial\nabla\phi} \cdot \mathbf{n}_{\mathcal{B}} = 0 \quad \text{on } \partial\mathcal{B} \quad (1.26)$$

$$Y := -\frac{\partial\psi}{\partial\phi} = -\frac{\partial\psi}{\partial g} \frac{\partial g}{\partial\phi} = -g'(\phi) \bar{Y}, \quad \bar{Y} := \frac{\partial\psi}{\partial g} \quad (1.27)$$

Here, Y is known as the conjugate energetic crack driving force and $\bar{Y}$ is known as the effective driving force which is related to the energetic degradation function.

1.4.2 Governing equations for dynamic fracture

In dynamic fracture an extra kinetic energy $\mathcal{K}$ is defined in terms of the velocity field $\dot{\mathbf{u}} := d\mathbf{u}/dt$ and damage rate field which is equivalent to phase-field parameter $\dot{\phi} := d\phi/dt$ that is,

$$\mathcal{K} = \int_{\Omega} \frac{1}{2} \rho \dot{\mathbf{u}} \cdot \dot{\mathbf{u}} dV + \int_{\mathcal{B}} \frac{1}{2} \varrho \dot{\phi} \cdot \dot{\phi} dV \quad (1.28)$$

where, ϱ has the dimension kg/m

It contributes to the total action defined by Lagrange, given by

$$\mathcal{A} := \int_{t_1}^{t_2} (\mathcal{E} - \mathcal{K}) dt \quad (1.29)$$

From Hamilton's principle, we have, $\delta\mathcal{A} = 0$

$$\begin{aligned}\delta\mathcal{A} = & \int_{t_1}^{t_2} \left[\int_{\Gamma} (\boldsymbol{\sigma} : \delta\boldsymbol{\epsilon} - \rho \dot{\mathbf{u}} \cdot \delta \dot{\mathbf{u}} dV) + \int_{\mathcal{B}} \left(G_c \left(\frac{\partial\gamma}{\partial\phi} \delta\phi + \frac{\partial\gamma}{\partial\nabla\phi} \cdot \delta\nabla\phi \right) + \frac{\partial\psi}{\partial\phi} \delta\phi - \varrho \dot{\phi} \cdot \delta\dot{\phi} \right) dV \right. \\ & \left. - \int_{\Omega} \mathbf{b}^* \cdot \delta \mathbf{u} dV - \int_{\partial\Omega_t} \mathbf{t}^* \cdot \delta \mathbf{u} dA \right] dt = 0\end{aligned}\quad (1.30)$$

$$\Rightarrow \frac{d}{dt} (\dot{\mathbf{u}} \cdot \delta \mathbf{u}) = \ddot{\mathbf{u}} \cdot \delta \mathbf{u} + \dot{\mathbf{u}} \cdot \delta \dot{\mathbf{u}} \quad (1.31)$$

Using the above equation and after rearrangement like before, we get,

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b}^* = \rho \ddot{\mathbf{u}} \quad \text{in } \Omega \quad (1.32)$$

$$Y - G_c \delta\phi\gamma \leq \varrho \ddot{\phi} \quad \text{in } \mathcal{B} \quad (1.33)$$

1.5 Geometric crack function

Till now, we have been defining crack surface density functional as some function of ϕ and $\nabla\phi$. Let's look at it in some greater detail.

1.5.1 Classical Geometric crack function

Consider a sharp crack located at $x = 0$ along a line to be represented as the following diffused or smeared presentation [22],

$$\phi(x) = \exp\left(-\frac{|x|}{l_0}\right) \quad (1.34)$$

having properties,

$$\phi(x=0) = 1, \quad \phi(x) \rightarrow 0 : x \rightarrow \pm\infty \quad (1.35)$$

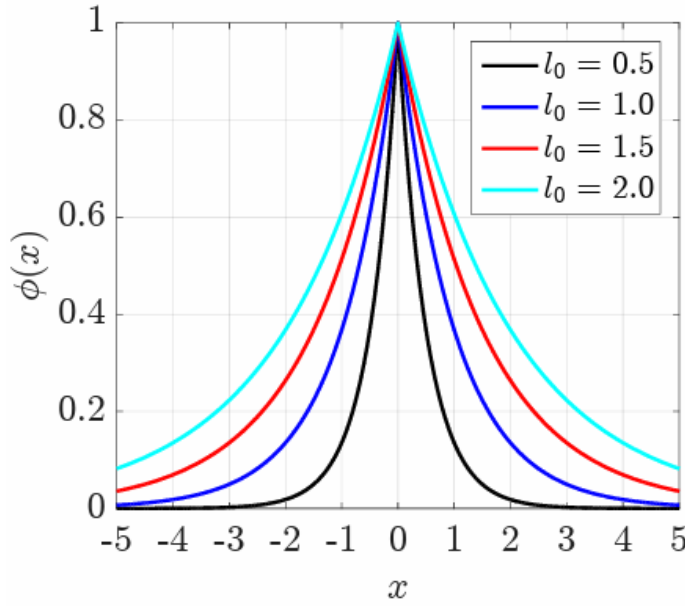


Figure 1.2: Classical geometric crack function with different values of l_0

This is one way to handle the discontinuity. If we look carefully at 1.34 it is actually the solution of this differential equation,

$$\frac{1}{l_0}\phi(x) - l_0 x''(x) = 0 \quad (1.36)$$

This brings us to the point that if we present our surface functional such that the minimization of it yields this equation, it will be consistent with our variational approach, hence

$$\Gamma_l(\phi) = \int_{-\infty}^{+\infty} \frac{1}{2} \left[\frac{1}{l_0} \phi^2 + l_0 (\phi')^2 \right] dx = \int_{-\infty}^{+\infty} \gamma(\phi; \nabla\phi) dx \quad (1.37)$$

here, ϕ^2 is our classical geometric function and l_0 is known as the length scale parameter. In the event of $l_0 \rightarrow 0$, the crack surface functional should become equal to the area of the crack. In this way, crack surface functional fulfills two roles.

This discussion was started for a 1-D case, but it can be easily extended to 2-D by this notation,

$$\gamma(\phi; \nabla\phi) = \frac{1}{2} \left[\frac{1}{l_0} \phi^2 + l_0 (\nabla\phi \cdot \nabla\phi) \right], \quad \delta_\phi \gamma = \frac{1}{l_0} \phi - l_0 \Delta\phi \quad (1.38)$$

1.5.2 Generic Geometric crack function

As mentioned earlier, instead of ϕ^2 , there are various other $\alpha(\phi)$ proposed by researchers, following that,

$$\gamma(\phi; \nabla \phi) = \frac{1}{c_0} \left[\frac{1}{l_0} \alpha(\phi) + l_0 |\nabla \phi|^2 \right], \quad \delta_\phi \gamma = \frac{1}{c_0} \left[\frac{1}{l_0} \alpha'(\phi) - l_0 \Delta \phi \right] \quad (1.39)$$

here, c_0 is known as the scaling parameter and it is obtained from the fact that on decreasing l_0 , crack surface functional should converge to the fully softened area of the crack,

$$c_0 := 4 \int_0^1 \alpha^{\frac{1}{2}}(\hat{\phi}) d\hat{\phi} \quad (1.40)$$

By change of variables, we can also conclude that,

$$x(\phi) = l_0 \int_0^1 \alpha^{-\frac{1}{2}}(\hat{\phi}) d\hat{\phi} \quad (1.41)$$

The generic crack geometric function is assumed to follow some properties, Wu [19] in his paper presented a polynomial that can generalize all the polynomials that can be taken,

$$\alpha(\phi) = \xi \phi + (1 - \xi) \phi^2 \in [0, 1] \quad \forall \phi \in [0, 1] \quad (1.42)$$

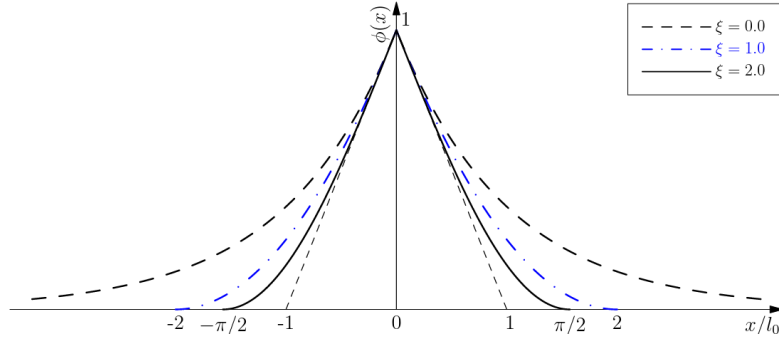


Figure 1.3: Generic geometric crack function

1.5.3 Length scale Parameter

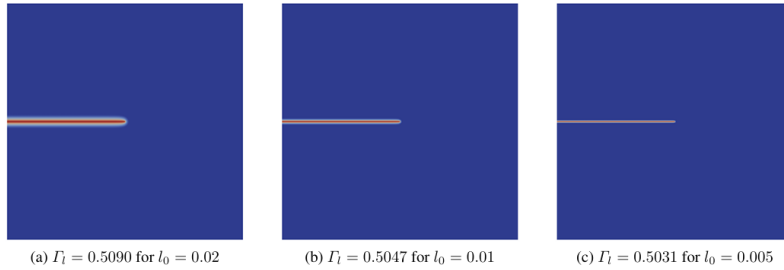


Figure 1.4: Variation of crack surface functional with l_0

The above images denote the calculated values of crack surface functional with decreasing l_0 . In this case, the actual crack surface area is 0.5 which is correctly achieved when we decrease the value of l_0 which is representative of the width of the crack. Decreasing l_0 might seem luring but it increases the computational costs a lot.

1.6 Stored energy functional splitting models

The strain energy density function for a linear isotropic elastic body is given by,

$$\psi_0(\boldsymbol{\epsilon}) = \frac{1}{2}\boldsymbol{\epsilon} : \mathbb{E}_0 : \boldsymbol{\epsilon} = \frac{1}{2}\lambda_0 \text{tr}^2(\boldsymbol{\epsilon}) + \mu_0 \boldsymbol{\epsilon} : \boldsymbol{\epsilon} = \frac{1}{2}\bar{\boldsymbol{\sigma}} : \mathbb{C}_0 : \bar{\boldsymbol{\sigma}} = \psi_0(\bar{\boldsymbol{\sigma}}) \quad (1.43)$$

where, the linear elasticity tensor $\mathbb{E}_0 = 2\mu_0\mathbb{I} + \lambda_0\mathbf{1} \otimes \mathbf{1}$ is defined in terms of the Lamé constants λ_0 and μ_0 , with $\mathbb{I}$ and $\mathbf{1}$ being the fourth- and second-order identity tensors, respectively; $\mathbb{C}_0 := \mathbb{E}_0^{-1}$ denotes the fourth-order compliance tensor; $\bar{\boldsymbol{\sigma}} = \mathbb{E}_0 : \boldsymbol{\epsilon}$ represents the effective stress tensor.

To prevent crack propagation in regions under compression, the strain energy density in (1.43) is split into a positive part and a negative part,

$$\psi_0(\boldsymbol{\epsilon}) = \psi_0^+(\boldsymbol{\epsilon}) + \psi_0^-(\boldsymbol{\epsilon}) \quad (1.44)$$

we consider only the tensile part of the initial strain energy to contribute to the growth of the crack, so the final strain energy density function is,

$$\psi(\boldsymbol{\epsilon}) = g(\phi)\psi_0^+(\boldsymbol{\epsilon}) + \psi_0^-(\boldsymbol{\epsilon}) \quad (1.45)$$

The resulting equilibrium equation and the phase field evolution hence become,

$$\boldsymbol{\sigma} = \frac{\partial \psi}{\partial \boldsymbol{\epsilon}} = g(\phi) \frac{\partial \psi_0^+}{\partial \boldsymbol{\epsilon}} + \frac{\partial \psi_0^-}{\partial \boldsymbol{\epsilon}} \quad (1.46)$$

$$G_c \delta_\phi \gamma = -g'(\phi) \bar{Y}, \quad \bar{Y} = \frac{\partial \psi}{\partial g} = \psi_0^+ \quad (1.47)$$

So, how should the splitting happen, and what should be the values of ψ_0^+ and ψ_0^- , have been discussed a lot among researchers. Below we have presented some of the most common models.

1.6.1 Isotropic model with no split [5]

For the isotropic model with no split, $\psi_0^- = 0$ and $\psi_0^+ = \psi_0$. Hence, the strain energy density is given by,

$$\psi(\boldsymbol{\epsilon}, \phi) = g(\phi)\psi_0(\boldsymbol{\epsilon}) \quad (1.48)$$

This results in the following stress field,

$$\boldsymbol{\sigma} = \frac{\partial \psi}{\partial \boldsymbol{\epsilon}} = g(\phi) \frac{\partial \psi_0}{\partial \boldsymbol{\epsilon}} = g(\phi) \bar{\boldsymbol{\sigma}}, \quad \bar{\boldsymbol{\sigma}} = \mathbb{E}_0 : \boldsymbol{\epsilon} \quad (1.49)$$

The above model cannot distinguish between the material under tension and compression. It predicts identical cracking under both tension and compression, which is not realistic for brittle quasi-static fracture.

1.6.2 Anisotropic model of Lancioni and Royer-Carfagni [11]

This model considers shear fracture. Specifically, only the deviatoric part of the initial strain energy density is affected by the degradation function, while the volumetric part remains unaffected. This is expressed as:

$$\psi_0^+ = \mu_0 \boldsymbol{\epsilon}_D : \boldsymbol{\epsilon}_D, \quad \psi_0^- = \frac{1}{2} K_0 \text{tr}^2(\boldsymbol{\epsilon}) \quad (1.50)$$

where, $K_0 = \lambda + 2\mu_0/3$ represents the bulk modulus. The strain tensor is decomposed into a volumetric part $\boldsymbol{\epsilon}_v$ and a deviatoric part $\boldsymbol{\epsilon}_d$,

$$\boldsymbol{\epsilon} = \boldsymbol{\epsilon}_v + \boldsymbol{\epsilon}_d \quad (1.51)$$

$$\boldsymbol{\epsilon}_v = \frac{1}{3} \text{tr}(\boldsymbol{\epsilon}) \mathbf{1}, \quad \boldsymbol{\epsilon}_d = \boldsymbol{\epsilon} - \frac{1}{3} \text{tr}(\boldsymbol{\epsilon}) \mathbf{1} \quad (1.52)$$

The resulting stress field in the Equation becomes:

$$\boldsymbol{\sigma} = g(\phi) 2\mu_0 \boldsymbol{\epsilon}_d + K_0 \text{tr}(\boldsymbol{\epsilon}) \mathbf{1} \quad (1.53)$$

This phase-field model (PFM) with the above split was employed to analyze the structural failures in some components of the French Panthéon, one of the most renowned historical landmarks in Paris.

1.6.3 Anisotropic model of Amor et al. [2]

This model made a slight change to cover a drawback of the previous model. In (1.53) even if the loading was tensile it was only covering the deviatoric part of total strain energy, due to which the stress was coming out more. Here, the split is as follows,

$$\psi_0^+ = \frac{1}{2}K_0\langle tr(\epsilon) \rangle^2 + \mu_0\epsilon_D : \epsilon_D, \quad \psi_0^- = \frac{1}{2}K_0\langle -tr(\epsilon) \rangle^2 \quad (1.54)$$

such that,

$$\sigma = g(\phi)[K_0\langle tr(\epsilon) \rangle \mathbf{1} + 2\mu_0\epsilon_d] - K_0\langle -tr(\epsilon) \rangle \mathbf{1} \quad (1.55)$$

where the Macaulay brackets $\langle \cdot \rangle$ are defined as $\langle x \rangle = (x + |x|)/2$

The above model partially limits but does not completely prevent crack propagation under compression. The deviatoric part of strain energy is still responsible for driving the crack when $tr(\epsilon) < 0$.

1.6.4 Anisotropic model of Miehe et al. [15]

This is also known as the strain-splitting model, where strains are divided into principal tensile and compressive parts

$$\epsilon = \sum_{n=1}^3 \epsilon_n \mathbf{p}_n \otimes \mathbf{p}_n = \epsilon^+ + \epsilon^- \quad (1.56)$$

$$\epsilon^+ = \sum_{n=1}^3 \langle \epsilon_n \rangle \mathbf{p}_n \otimes \mathbf{p}_n, \quad \epsilon^- = \sum_{n=1}^3 -\langle -\epsilon_n \rangle \mathbf{p}_n \otimes \mathbf{p}_n \quad (1.57)$$

$$\psi_0^\pm(\epsilon) = \frac{1}{2}\lambda_0\langle \pm tr(\epsilon) \rangle^2 + \mu_0\epsilon^\pm : \epsilon^\pm \quad (1.58)$$

It follows that,

$$\sigma = g(\phi)[\lambda_0\langle tr(\epsilon) \rangle \mathbf{1} + 2\mu_0\epsilon^+] - \lambda_0\langle -tr(\epsilon) \rangle \mathbf{1} + 2\mu_0\epsilon^- \quad (1.59)$$

This PFM can eliminate the creation of cracks under compression

1.6.5 Anisotropic model of Wu and Nguyen [20]

Similar to the above, this is known as the stress-splitting model. Here principal stresses are used in place of principal strains

$$\bar{\sigma} = \bar{\sigma}^+ + \bar{\sigma}^-, \quad \bar{\sigma}^\pm = \sum_{n=1}^3 \bar{\sigma}^\pm \mathbf{p}_n \otimes \mathbf{p}_n \quad (1.60)$$

$$\psi_0^\pm(\bar{\sigma}) = \frac{1}{2}\bar{\sigma}^\pm : \mathbb{C}_0 : \bar{\sigma}^\pm \quad (1.61)$$

$$\frac{\partial \psi_0^\pm}{\partial \epsilon} = \mathbb{E}_0 : \frac{\partial \psi_0^\pm}{\partial \bar{\sigma}} = \bar{\sigma}^\pm, \quad \sigma = g(\phi)\bar{\sigma}^+ + \bar{\sigma}^- \quad (1.62)$$

This anisotropic PFM was proposed for both brittle and quasi-brittle fracture

1.7 Energetic degradation function

In phase field models, $g(\phi) : [0, 1] \rightarrow [1, 0]$ is called energetic degradation function. It plays an important role since it links the crack phase-field and mechanical fields. It determines how stored energy functional responds to changes in the crack phase-field. It is only attached to the positive part of strain energy as mentioned earlier. The energetic degradation function has to satisfy the following properties:

1. In the fully intact region i.e. $\phi = 0$ strain energy density of material should be equal to the strain energy density of linear isotropic elastic material as given in (1.44) and (1.43). Hence,

$$g(0) = 1 \quad (1.63)$$

2. In the fully broken region, $\phi = 1$, strain energy due to tension is released and converted to surface energy for the creation of a new surface as a crack. Hence the positive strain energy should be zero. Therefore,

$$g(1) = 0 \quad (1.64)$$

3. From the above two equations we can remark that the value of $g(\phi)$ is an indicator of the crack growth hence, as ϕ increases it should always move towards 0. No recovery is allowed so it follows that $g(\phi)$ should be a monotonically decreasing function, or

$$g'(\phi) < 0 \quad (1.65)$$

4. In the fully broken state, $\phi = 1$ it follows from the crack evolution law that the crack driving force that is represented by $g'(\phi)\psi_0^+$ should become zero. There should be no more growth in any direction. Hence,

$$g'(1) = 0 \quad (1.66)$$

There is a family of curves that satisfy the above properties but the simplest and most widely used quadratic degradation function is given by

$$g(\phi) = (1 - \phi^2) \quad (1.67)$$

A small constant k is added to the degradation function for numerical stability. Therefore, 1.45 can be re-written as:

$$\psi(\epsilon(\mathbf{u}), \phi) = ((1 - \phi^2) + k)\psi_0^+(\epsilon) + \psi_0^-(\epsilon) \quad (1.68)$$

1.8 Governing equations in weak form

Given the strong form of differential equations that we have already derived above, we can easily find the weak form of our equations using the Weighted Residual approach. Firstly, for the displacement sub-problem,

$$\begin{aligned} \int_{\Omega} (\nabla \cdot \boldsymbol{\sigma} + \mathbf{b}^* - \rho \ddot{\mathbf{u}}) \delta \mathbf{u} dV &= 0 \quad (1.69) \\ \int_{\Omega} \frac{\partial \boldsymbol{\sigma}_{ij}}{\partial x_j} : \delta \mathbf{u}_i dV + \int_{\Omega} \mathbf{b}^* \cdot \delta \mathbf{u} dV - \int_{\Omega} \rho \ddot{\mathbf{u}} \cdot \delta \mathbf{u} dV &= 0 \\ \int_{\Omega} \frac{\partial}{\partial x_j} (\boldsymbol{\sigma}_{ij} \delta \mathbf{u}_i) dV - \int_{\Omega} \boldsymbol{\sigma}_{ij} \frac{\partial}{\partial x_j} (\delta \mathbf{u}_i) dV + \int_{\Omega} \mathbf{b}^* \cdot \delta \mathbf{u} dV - \int_{\Omega} \rho \ddot{\mathbf{u}} \cdot \delta \mathbf{u} dV &= 0 \\ \int_{\partial \Omega_t} (\boldsymbol{\sigma}_{ij} n_j) \delta \mathbf{u}_i dA + \int_{\Omega} \mathbf{b}^* \cdot \delta \mathbf{u} dV &= \int_{\Omega} \boldsymbol{\sigma}_{ij} \frac{\partial}{\partial x_j} (\delta \mathbf{u}_i) dV + \int_{\Omega} \rho \ddot{\mathbf{u}} \cdot \delta \mathbf{u} dV \\ \int_{\Omega} \boldsymbol{\sigma} : \nabla^s \delta \mathbf{u} dV + \int_{\Omega} \rho \ddot{\mathbf{u}} \cdot \delta \mathbf{u} dV &= \delta \mathcal{P} \quad \forall \delta \mathbf{u} \in \mathcal{V}_u \end{aligned} \quad (1.70)$$

where, $\delta \mathcal{P}$ is virtual power of body and surface forces

For the phase-field parameter,

$$\int_{\mathcal{B}} (g'(\phi) \bar{Y} + G_c \delta \phi \gamma) \delta \phi dV \geq 0 \quad (1.71)$$

$$\int_{\mathcal{B}} (g'(\phi) \bar{Y} \delta \phi + G_c \delta \gamma) dV \geq 0 \quad \forall \delta \phi \in \mathcal{V}_{\phi} \quad (1.72)$$

We must also note the trial spaces as follows,

$$\begin{aligned} \mathcal{U}_u &:= \{\mathbf{u} | \mathbf{u}(\mathbf{x}) = \mathbf{u}^* \ \forall \mathbf{x} \in \partial \Omega_u\}, & \mathcal{V}_u &:= \{\delta \mathbf{u} | \delta \mathbf{u}(\mathbf{x}) = \mathbf{0} \ \forall \mathbf{x} \in \partial \Omega_u\} \\ \mathcal{U}_{\phi} &:= \{\phi | \phi(\mathbf{x}) \in [0, 1], \dot{\phi}(\mathbf{x}) \geq 0 \ \forall \mathbf{x} \in \mathcal{B}\}, & \mathcal{V}_{\phi} &:= \{\delta \phi | \delta \phi(\mathbf{x}) \geq 0 \ \forall \mathbf{x} \in \mathcal{B}\} \end{aligned} \quad (1.73)$$

Chapter 2

Using MOOSE for Solid Mechanics problems

2.1 Introduction

Multiphysics Object-Oriented Simulation Environment (MOOSE) is a software that is used to solve physical problems and systems involving mathematical equations using the finite element framework. For this purpose, it has various built-in modules with extensive tutorials and example input files for users to understand and modify accordingly to get their results. It also has the feature of application development where users get the option of using the modules of their choice with their self-developed weak forms of governing differential equations, material behavior characteristics, etc.

As part of this project, we have first tried to develop an understanding of the environment by going through heat transfer and tensor mechanics modules respectively. For heat conduction and diffusion problems, we have also developed our application and compared the results with the existing module. In this section, we have covered how to solve some standard quasi-static and dynamic solid mechanics problems using MOOSE. More examples and problems including the heat transfer module are given in the appendix.

2.2 Uni-axial loading (Quasi-Static)

A 2D plane stress element is subjected to a uni-axial state of stress, $\sigma_{11} = 1$. $E = 1000$ and $\nu = 0.25$. From Hooke's law, we can find the strain component as follows,

$$\epsilon_{11} = \frac{\sigma_{11}}{E} = 10^{-3}, \quad u_1 = 10^{-3} \quad (2.1)$$

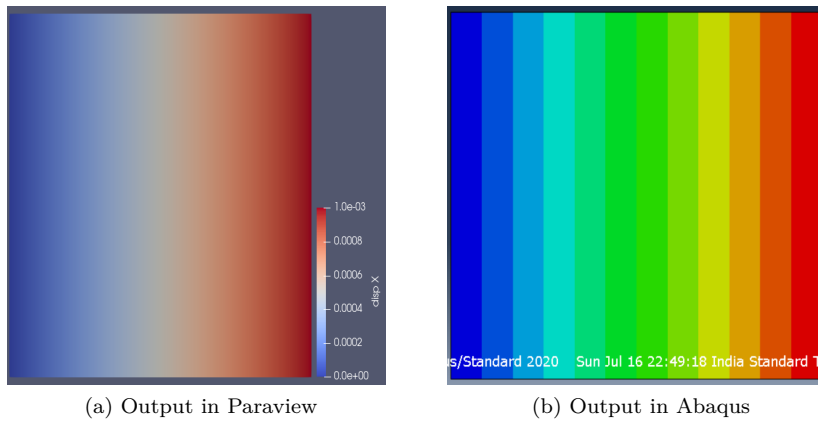


Figure 2.1: Representation of displacement

Let's look at the input file required to run this problem in MOOSE

Variables

```
[GlobalParams]
  displacements = 'disp_x disp_y'
  out_of_plane_strain = strain_zz
[]

[Variables]
  [disp_x]
    order = FIRST
    family = LAGRANGE
  []
  [disp_y]
    order = FIRST
    family = LAGRANGE
  []
  [strain_zz]
  []
[]
```

Figure 2.2: variables change

disp_x and disp_y are declared as separate variables in Tensor mechanics problems signifying to the software it is a 2-D problem. out_of_plane is also provided when you want to do a 2-D problem but want to specify the strain_zz present there because of plane stress condition.

Kernels

```
[Modules/TensorMechanics/Master]
[all]
  planar_formulation= WEAK_PLANE_STRESS
  strain= SMALL
  add_variables = true
  generate_output='vonmises_stress stress_xx stress_yy stress_xy'
[]
[]
```

Figure 2.3: Master action in Tensor Mechanics

In the Tensor mechanics module, for the kernel formation, the Master action passes all the variables to the stress divergence and all the associated kernels that are required and linked with each other. We can define the space and time kernels separately for each variable as well if we do not use the Master action.

In addition to this, we can also make our kernel by creating required *.C* and *.h* files and storing them where the default kernel files are stored. This can also be referred to in the Appendix for more detail.

BCs

```
[Pressure]
[rightl]
  boundary = right
  function = -1
[]
[]
```

Figure 2.4: Applying pressure in Boundary Condition block

BCs are applied similarly to the heat transfer module. For pressure, we give the value as a function

Materials

```
[Materials]
[Elasticity_tensor]
  type = ComputeIsotropicElasticityTensor
  youngs_modulus = 1000
  poissons_ratio=0.25
[]
[stress]
  type = ComputeLinearElasticStress
[]
[]
```

Figure 2.5: Materials block

Here we make the elasticity tensor. If there is any behaviour change like we want to introduce plasticity etc., it would be accomplished from this block only.

2.3 Thermo-mechanical problem

In this case, again we have considered a 2D Plane Stress element. It is initially at 0 K. A temperature step function is applied which raises its temperature by 5 K. The dimensions of the element are 1×1 and its coefficient of thermal expansion = 1. $E = 1$ and $\nu = 0.1$.

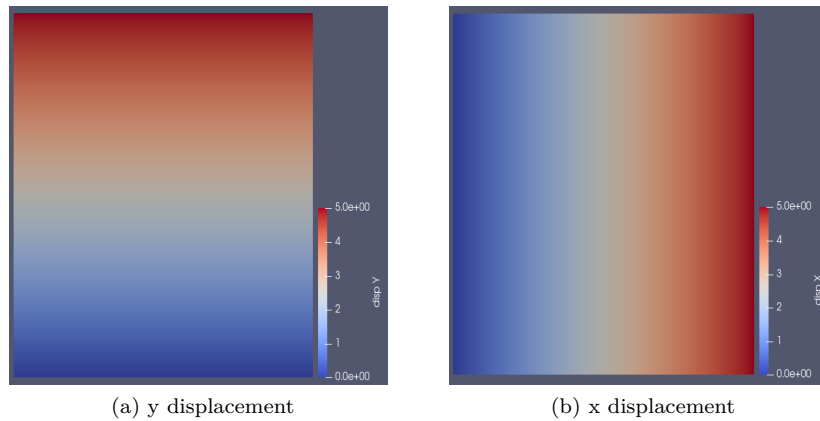


Figure 2.6: 2D Plane Stress element undergoing thermal expansion

As expected both sides get expanded by $L\alpha\nabla T = 5$ units and there is no stress.

```
[expansion1]
  type = ComputeThermalExpansionEigenstrain
  temperature = T
  thermal_expansion_coeff = 1
  stress_free_temperature = 0
  eigenstrain_name = thermal_expansion
[]
```

Figure 2.7: Changes in material block

This part is added in the Materials block along with the declaration of temperature as an auxiliary variable as shown here.

```

[AuxVariables]
  [T]
  []
[]

[AuxKernels]
  [temperature_ramp]
    type = FunctionAux
    execute_on = TIMESTEP_BEGIN
    variable = T
    function = 5
  []
[]

```

Figure 2.8: AuxKernel block

2.4 Uni-axial loading (Dynamic) [18]

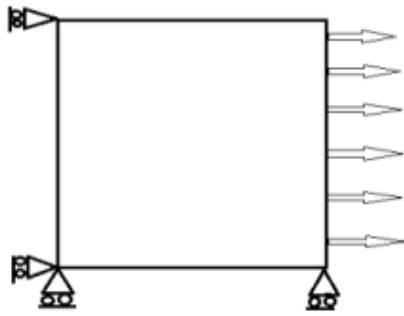


Figure 2.9: Geometric and Boundary conditions for the dynamic plane strain problem

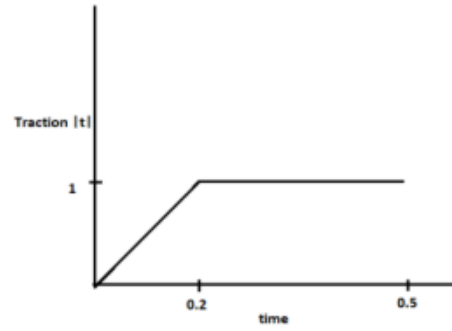


Figure 2.10: Rate of traction applied

We consider a 2D plane strain problem. The analysis here is transient. We have solved this problem using the implicit Newmark-Beta method in MOOSE and, using both explicit and implicit methods in Abaqus. The dimensions of the element are 1×1 . $E = 1000$ MPa, $\nu = 0.3$ and $\rho = 1$ kg/mm³.

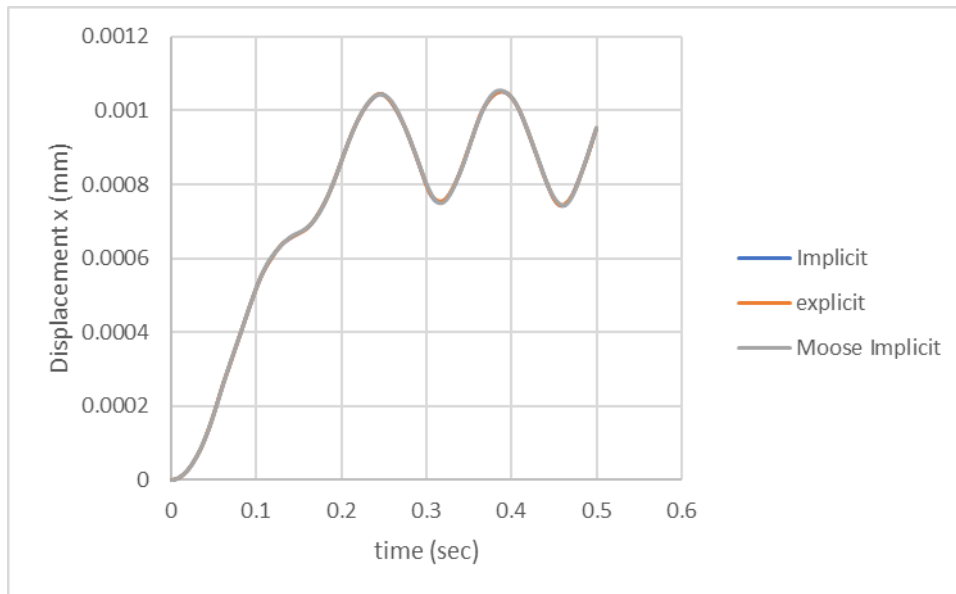


Figure 2.11: Displacement profile comparison

Displacement profiles match with Ansys for us which indicates the problem is solved fairly correctly.

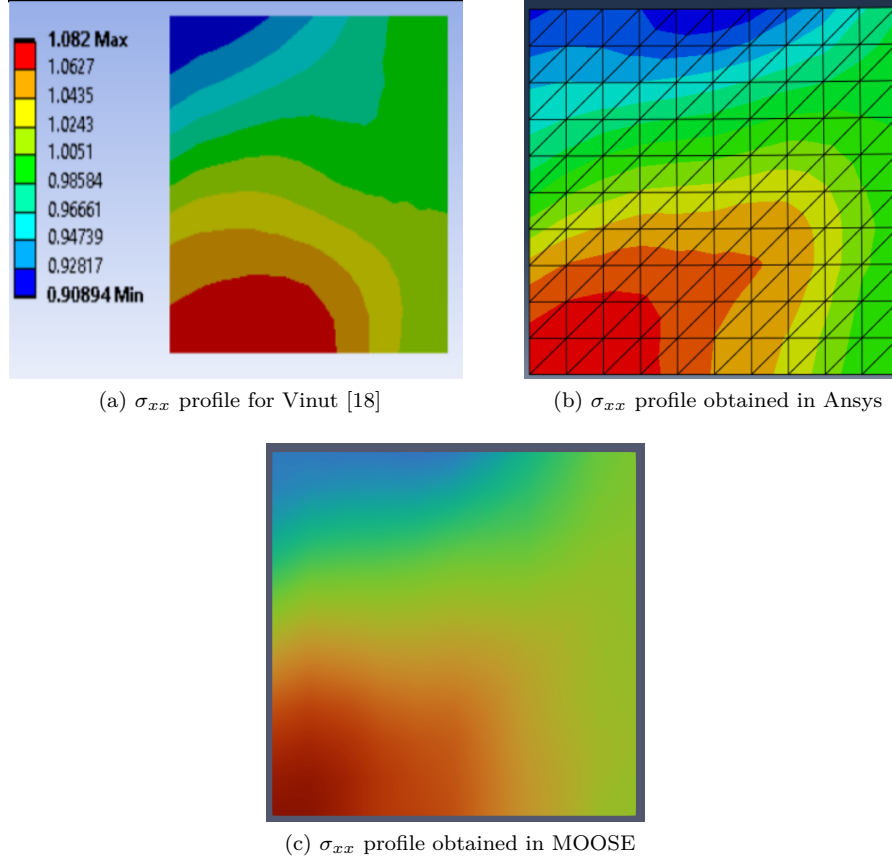


Figure 2.12: Comparing σ_{xx} profiles obtained from different sources

The resulting patterns of stress are similar however, there is a slight difference in maximum values for the three cases. For Vinut [18] it is 1.082 MPa, however, for Ansys we obtained 1.053, and for MOOSE we obtained 1.06746.

```
[Modules/TensorMechanics/DynamicMaster]
[all]
  add_variables = true
  newmark_beta = 0.25
  newmark_gamma = 0.5
  strain = SMALL
  density = 1
  generate_output = 'stress_xx stress_xy stress_yy'
  mass_damping_coefficient = 0
  stiffness_damping_coefficient = 0
  hht_alpha = 0
  use_automatic_differentiation = true
  save_in = 'fx fy'
[]
[]
```

Figure 2.13: Adding Dynamic Master Action in MOOSE

For earlier problems, we have shown how master action can add all the required kernels in a solid mechanics problem. That works for quasi-static problems. For dynamic problems, we have to use the dynamic master action which also allows us to choose Newmark-Beta values along with many other parameters. If we had to do this addition manually, we have to add inertial kernels for all the related displacements and then define how through AuxKernels,

velocity and acceleration are calculated for subsequent steps using Newmark-Beta pr other parameters.

```
[Functions]
[right_pull]
  type = PiecewiseLinear
  x = '0 0.2 0.5'
  y = '0 1 1'
[]
[]
```

Figure 2.14: Creating a linear piece-wise function

The varying traction load of this problem can be applied as pressure with the above variation and *scalefactor* = -1

```
[Executioner]
  type = Transient

  solve_type = NEWTON
  petsc_options_iname = '-pc_type -pc_factor_mat_solver_package'
  petsc_options_value = 'lu          superlu_dist'
  automatic_scaling = true

  nl_rel_tol = 1e-8
  nl_abs_tol = 1e-10

  dt = 0.005
  end_time = 0.5
[]
```

Figure 2.15: Executioner Block

For the transient problem, here we can define the time step, start time, and end time required for the analysis. Using *scheme* = option also allows the use of Explicit-Euler or other Time Integrator systems. The default is Implicit-Euler which is used by us also.

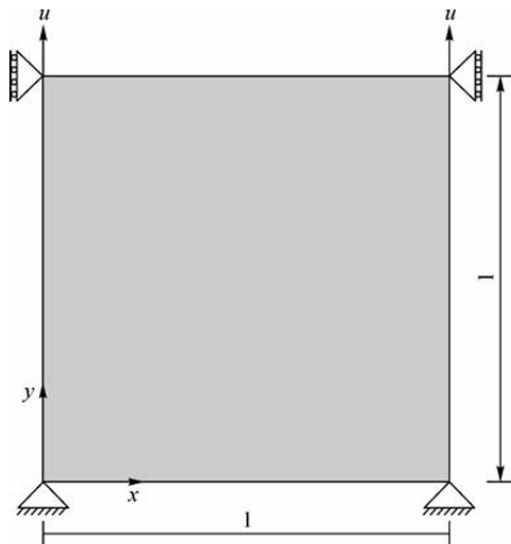
Chapter 3

Using MOOSE for benchmark Quasi-Static Phase-field problems

3.1 Introduction

Before this chapter, we saw how MOOSE can be used to solve solid mechanics problems using its modules. In this section, we will look at how to modify our code to solve the phase field part. In other words, how do we solve the combined problem of solid mechanics and phase field? There are two ways for this. Using the given phase field module or the staggered scheme that is implemented using the Multi-app feature in MOOSE. For the first problem, we compared our results using both methods and after getting almost exact results, we used the staggered scheme for all the remaining problems.

3.2 Uni-axial tension applied to a plane strain domain [16]



Property	Value
G_c	$5 \times 10^{-3} \text{ kN/mm}$
E	210 kN/mm^2
ν	0.3
l	0.1 mm
δu	$10^{-3} \text{ mm (linear)}$

Table 3.1: Material Properties

Figure 3.1: Geometric and Boundary conditions for single element

In this section, we have considered a rectangular element that is subjected to displacement boundary conditions. Initially, the whole element is uncracked that is $\phi = 0$. There is no

crack presumed anywhere. We try to find out that as the displacement is applied what is the change in the phase field parameter throughout the body. The properties of the element are listed in the following table. The top edge is displaced to $u = 0.1 \text{ mm}$.

3.2.1 Analytical Solution

Given the displacement field, we can find the strain tensor and then the Cauchy stress tensor is given by,

$$\bar{\sigma} = g(\phi)2\mu_0\epsilon + \lambda_0 \text{tr}(\epsilon)\mathbf{1} \quad (3.1)$$

Stresses come out homogeneous because of the given boundary conditions. The phase field parameter will also be homogeneous.

$$\bar{\sigma}_{yy} = \frac{E(1-\nu)}{(1+\nu)(1-2\nu)}\epsilon_{yy} \quad (3.2)$$

We are not calculating other components because only $\bar{\sigma}_{yy}$ contributes to strain energy which is given by,

$$\psi_0 = \frac{1}{2} \frac{E(1-\nu)}{(1+\nu)(1-2\nu)}\epsilon_{yy}^2 \quad (3.3)$$

The equilibrium equation is identically satisfied, we need to go through the phase-field evolution equation to find ϕ , i.e.,

$$G_c \delta_\phi \gamma = -g'(\phi)\bar{Y} \quad (3.4)$$

Consider $\alpha = \phi^2$ and $g(\phi) = (1-\phi)^2 + k$ we have,

$$\frac{G_c}{l_0}\phi - G_c l_0 \nabla^2 \phi = -2(1-\phi)\bar{Y} \quad (3.5)$$

Consider $\bar{Y} = \psi_0^+ = \psi_0$ as we have only tensile strain energy and $\nabla^2 \phi = 0$ because of the homogeneous problem. After one step we get,

$$\phi = \frac{2\psi_0}{\frac{G_c}{l_0} + 2\psi_0} = \frac{\frac{E(1-\nu)}{(1+\nu)(1-2\nu)}\epsilon_{yy}^2}{\frac{G_c}{l_0} + \frac{E(1-\nu)}{(1+\nu)(1-2\nu)}\epsilon_{yy}^2} \quad (3.6)$$

$$\sigma_{yy} = g(\phi)\bar{\sigma}_{yy} = (1-\phi)^2\bar{\sigma}_{yy} \quad (3.7)$$

3.2.2 Solution using phase field module

The finite element formulation of our benchmark problem required two coupled partial differential equations to be solved. In the combined module of MOOSE, we were able to find some tests that use the `\Master` action from the Tensor Mechanics module and `\Nonconserved` action from the Phase-field module. The input file of the code is provided in the appendix.

Important points to note are:

1. The phase-field evolution law that we want to solve is presented as damage evolution law, with damage parameter d changing with time.

$$\dot{d} = -L \frac{\delta F}{\delta d} = -L \left(\frac{\partial f_{loc}}{\partial d} - \nabla \cdot \kappa \nabla d \right), \quad (3.8)$$

where, $L = l_0/(\eta G_c)$, $\kappa = G_c l_0$ and $\eta = \text{viscosity parameter which is taken as } 10^{-20} \approx \text{zero for our analysis}$

Also, Wu [19] in his paper has proved that damage evolution law coincides with Phase-field, so we can use that.

2. One thing that we observed by seeing output plots of parameters with respect to the time in tutorial problems of MOOSE is the two-step formulation technique adopted in solving. For instance, if we have kept displacement of top boundary = 0.02 mm, first it will take $\phi = 0$, insert it in the equilibrium equation, solve for u , find stress tensor, and then insert the obtained strain energy into the phase field evolution law to predict the current phase field parameter. This correspondence was important to note when we tried to implement the time-dependent load.
3. As a result of that observation and as a first step for the problem formulation we started by giving a fixed boundary condition to the problem, running it for two steps, and then noting the result. For the next iteration or displacement, we will change the boundary condition by the increment and again run the file. In this way, we noted the data for 10 points and ensured that it is matching with the standard results.
4. We also tried giving the displacement load as a function of time but, we faced a correspondence error. We were getting correct phase field parameter values one step later and stress values were wrong. We used `use_current_history_variable = true` in the `ComputeLinearElasticPFFractureStress` material action to correct this error.
5. In addition to this we also had a look at solver options and realized that for enforcing the condition that the phase field variable can't decrease its value and the maximum value it can have is 1, we added `use_snes_vt_solver = true` in the same material action mentioned above and changed the solver as shown in the following figure. The full code for this problem can be found in the Appendix.

```
[Executioner]
  type = Transient

  solve_type = PJFNK
  petsc_options_iname = '-pc_type -ksp_gmres_restart -sub_ksp_type -sub_pc_type -pc_asm_overlap'
  petsc_options_value = 'asm 31 preonly lu 1'

  nl_rel_tol = 1e-8
  l_max_its = 10
  nl_max_its = 10

  dt = 1e-4
  dtmin = 1e-4
  num_steps = 2
[]
```

Figure 3.2: moose solver

```
[Executioner]
  type = Transient

  solve_type = PJFNK
  petsc_options_iname = '-pc_type -snes_type'
  petsc_options_value = 'lu vinewtonrsls'

  nl_rel_tol = 1e-8
  l_max_its = 10
  nl_max_its = 10

  dt = 1e-3
  dtmin = 1e-3
  num_steps = 100
[]
```

Figure 3.3: our solver

3.2.3 Solution using Staggered scheme

In the Phase field module, we saw the use of the Allen-Cahn kernel and the damage evolution law and used it based upon the similarity with the phase field evolution law making mobility very small. Also for the solver part, the phase field module solves the displacement and phase

field variables monolithically (together in one step) which can be computationally expensive for problems having higher degrees of freedom.

Staggered scheme is considered a very useful method for solving coupled partial differential equations. In MOOSE we have the capability of MultiApps through which we can realize it.

MultiApps in MOOSE

In MultiApp we can have two different files with their individual solvers linked to each other which allows the transfer of data from one file to the other. The first file which is usually called the parent file or source file can be considered as the main solve which requires some of its auxiliary variables to be solved separately. These auxiliary variables are primary variables in the child file. Files can exchange post-processor values or variable values directly as required. Based upon the exchange there are two types of coupling defined in MultiApps in MOOSE.

1. Loose Coupling: In this, the exchange happens only once per time step. Loose coupling, though easy and stable, may not be accurate. Since data is only exchanged once per time step there is the very real possibility that the solutions in both apps are not at an equilibrium with each other. This is especially true as timesteps grow larger.
2. Tight Coupling (Picard iteration): Here the exchange keeps happening till a convergence criterion is reached for the main solve or in simple terms the value of the objective variable stops changing with more iterations. To get this behavior with MultiApps, all that is needed is to set *fixed_point_max_its* in the Executioner block of the parent app to greater than 1.

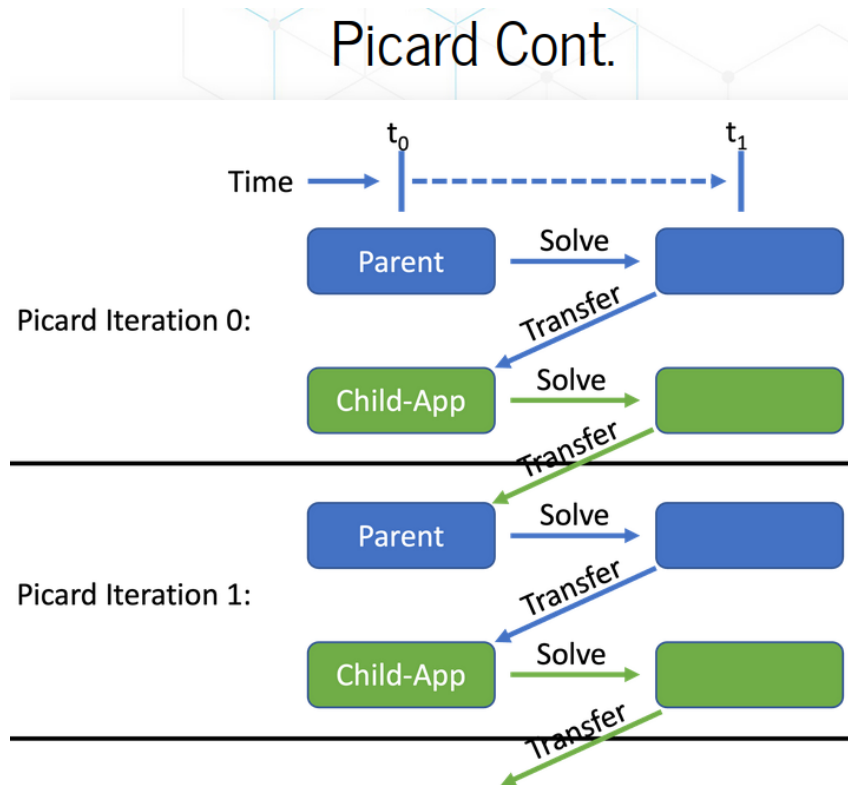


Figure 3.4: Picard iterations

Staggered scheme for the Phase Field Method

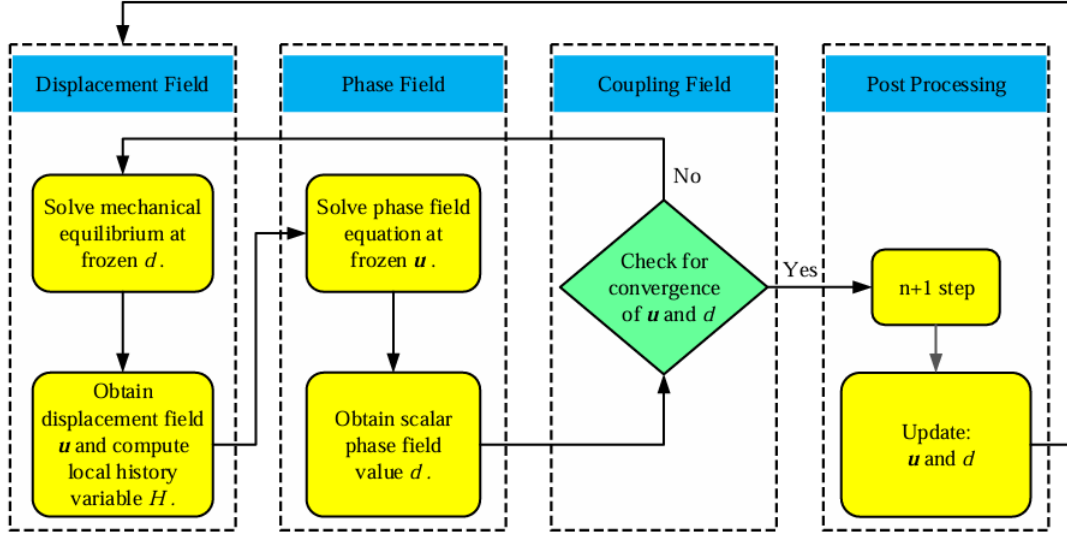


Figure 3.5: Flow chart for staggered scheme

Let's look at the two files that we need to create block by block to understand how the staggered scheme is approached in MOOSE. Files we have made are adapted from the RACCOON app made by Hugary [7] who is a contributor to the MOOSE environment. We have installed the RACCOON app in our MOOSE environment and then we can run these files after making appropriate changes for our problems. First, we will have a look at the parent file for this uniaxial plane strain field problem.

elasticity.i

```

E = 210
nu = 0.3
K = '${fparse E/3/(1-2*nu)}'
G = '${fparse E/2/(1+nu)}'

Gc = 5e-3
l = 0.1

```

The input file starts with defining the material properties in which special attention should be given to the units used. Here E is in KN/mm^2 , G_c is in KN/mm and l is in mm . So, later when the load is calculated it is in KN only.

K and G are the Bulk Modulus and the Shear Modulus of the material respectively which will be required in further sections.

```

[MultiApps]
  [fracture]
    type = TransientMultiApp
    positions = '0 0 0'
    input_files = 'fracture_rushi.i'
    cli_args = 'Gc=${Gc};l=${l} '
    execute_on = 'Timestep_End'
  []
[]

[Transfers]
  [from_d]
    type = MultiAppCopyTransfer
    multi_app = fracture
    direction = from_multiapp
    variable = d
    source_variable = d
  []
  [to_psie_active]
    type = MultiAppCopyTransfer
    multi_app = fracture
    direction = to_multiapp
    variable = 'psie_active fy'
    source_variable = 'psie_active fy'
  []
[]

```

This is the block where the actual linking happens. The input file that we have mentioned should be in the same folder as elasticity.i. The value given in the positions signifies that the origin is the same for both files. Then we pass some scalar values that we want to use in the child file as well. After that, we defined that the fracture app (child app) should execute at the end of the timestep of the parent app. There are other options for this execute_on command as well such as initial, timestep.begin, linear, nonlinear, MultiApp_fixedpoint_end as well which are used as per requirement.

In Transfers action we define the control sequence of the variables. MultiAppCopyTransfer is used to transfer variable values at every mesh point. Again there are other options such as MultiAppPostprocessorTransfer as well.

After this we define the mesh which follows the same procedure for all modules using GeneratedMeshGenerator.

```

[AuxVariables]
  [fy]
  []
  [d]
  []
[]

[Kernels]
  [solid_x]
    type = ADStressDivergenceTensors
    variable = disp_x
    component = 0
  []
  [solid_y]
    type = ADStressDivergenceTensors
    variable = disp_y
    component = 1
    save_in = fy
  []
[]

```

The equation that we want to solve in the parent file is the equilibrium equation,

$$\nabla \cdot \boldsymbol{\sigma} = 0 \quad (3.9)$$

Kernels are added for that and f_y and d are declared as auxiliary variables which will be calculated using the primary variables for this file, which are disp_x and disp_y . Following this BCs are defined like we used to define for the Solid Mechanics app.

```
[Materials]
[bulk]
  type = ADGenericConstantMaterial
  prop_names = 'K G'
  prop_values = '${K} ${G}'
[]
[degradation]
  type = PowerDegradationFunction
  f_name = g
  function = (1-d)^p*(1-eta)+eta
  phase_field = d
  parameter_names = 'p eta '
  parameter_values = '2 1e-6'
[]
[strain]
  type = ADComputeSmallStrain
[]
```

Material properties are defined for every quadrature point along with the degradation function defined as $g(\phi)$ in the literature in this block. Strain is also calculated. (This is done using a fixed value of phase field value initialized or obtained from the previous step).

```
[elasticity]
  type = SmallDeformationIsotropicElasticity
  bulk_modulus = K
  shear_modulus = G
  phase_field = d
  degradation_function = g
  decomposition = NONE
  output_properties = 'elastic_strain psie_active'
  outputs = exodus
[]
[stress]
  type = ComputeSmallDeformationStress
  elasticity_model = elasticity
  output_properties = 'stress'
  outputs = exodus
[]
[]
```

SmallDeformationIsotropicElasticity.C is the source file that takes the strain and other values obtained till now and calculates the total elastic strain energy and the tensile or the active part of the strain energy which is also transferred to the fracture file for its use. We can select the decomposition method we require such as strain spectral here, but because only tensile stresses are present in this problem for us we have kept the option of no decomposition.

Deformed stress values are also calculated after this step and all the components are stored for displaying in output.

```

[Postprocessors]
[Fy]
    type = NodalSum
    variable = fy
    boundary = top
    execute_on = 'initial timestep_end'
[]
[phi]
    type = PointValue
    point = '0.5 0.5 0'
    variable = d
    execute_on = 'initial timestep_end'
[]
[./average_psi]
    type = ElementAverageValue
    variable = psie_active
[../]

```

We now report the sum of the values of the residual at the top boundary at the start of the program and after every timestep of the parent file which is equal to the reaction force. We have only one element so we have reported the phase field parameter at the center point (we could have taken any other point, the whole element is like a point). The element average value of the active strain energy is also reported.

```

[./change_over_time1]
    type = ChangeOverFixedPointPostprocessor
    postprocessor = 'Fy'
    change_with_respect_to_initial = false
    execute_on = 'initial timestep_end'
[../]
[./change_over_time2]
    type = ChangeOverFixedPointPostprocessor
    postprocessor = 'phi'
    change_with_respect_to_initial = false
    execute_on = 'initial timestep_end'
[../]

```

We also measured the change in the postprocessor values before beginning the iteration and at the last iteration when the timestep ends. If the values come out zero it means in the last iteration no change took place in the variable values which means the number of iterations or the solution method is correct. This is a very important metric, especially in large problems when the expected profile doesn't match with our solution and we need to find out at which point we need to change the execution parameters.

```

[Executioner]
    type = Transient

    solve_type = NEWTON
    petsc_options_iname = '-pc_type -pc_factor_mat_solver_package'
    petsc_options_value = 'lu          superlu_dist'
    automatic_scaling = true

    nl_rel_tol = 1e-8
    nl_abs_tol = 1e-10

    dt = 0.001
    end_time = 0.1

    fixed_point_max_its = 20
    fixed_point_min_its = 2
    accept_on_max_fixed_point_iteration = true
    fixed_point_rel_tol = 1e-8
    fixed_point_abs_tol = 1e-10
[]

```

After defining the complete problem we need to define using which method the final equations are to be solved. In our case, we are using the Newton_Raphson iteration technique. RACCOON app tutorials also use the same solver.

The parameters in this block play a very crucial role in the solution of the phase field problem. However, from this particular problem, we added a line in this Executioner block without which again we were getting the correspondence error we got while using the phase field module.

fixed_point_min_its = 2

Fixed point iterations is the name used for Picard iterations i.e. how many times the transfer takes place between the parent and the child app. At the zeroth iteration, all the values are initialized in both files. Time is used only as a load parameter here, the problem is essentially quasi-static only. So for every iteration, there is a step load increment of 0.001 at the top till the load reaches 0.1 mm. The nonlinear absolute tolerance for the parent app is set to 10^{-10} . This tolerance is the global L2 norm of the residual of the equilibrium equation that the parent app is solving. The fixed point absolute tolerance is also set to 10^{-10} . For every Picard iteration, the parent app and child app have their non-linear iterations whose convergence criterion we just wrote. For the Picard iterations to stop or converge and move to the next load step, the parent app's global L2 norm of the residual must be less than the fixed point absolute tolerance in the first non-linear iteration itself.

But here we have a crucial point. The solver checks that the fixed point tolerance criterion is met and moves on to the next step without updating the effect the variables would have on receiving a new phase field parameter value. In the Post processor output also, it reports the values of the variables at the previous Picard iteration only. Hence, for this error, we had to set fixed_point_min_its at least 2 so that the change is reported. We can have a look at the following output of the elasticity run for the first time step to understand more.

```

0 Picard |R| = 1.414214e-03
1 Picard |R| = 0.000000e+00
2 Picard |R| = 0.000000e+00
Fixed point convergence reason: CONVERGED_ABS
Postprocessor Values:
+-----+-----+-----+-----+-----+-----+
| time | Fy | average_psl | change_over_time1 | change_over_time2 | phi |
+-----+-----+-----+-----+-----+-----+
| 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 |
| 1.000000e-03 | 2.795226e-01 | 1.413462e-04 | -3.169685e-03 | 5.622054e-03 | 5.622054e-03 |
+-----+-----+-----+-----+-----+-----+
Advancing MultiApps on TIMESTEP_END
Fracture0:
Fracture0: Outlier Variable Residual Norms:
Fracture0: d: 5.824190e-18
Fracture0:
Fracture0: Postprocessor Values:
+-----+-----+-----+-----+-----+-----+
Fracture0: | time | Fy | average_psl | change_over_time | change_over_time2 | phi |
Fracture0: +-----+-----+-----+-----+-----+-----+
Fracture0: | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 | 0.000000e+00 |
Fracture0: | 1.000000e-03 | 2.795226e-01 | 1.413462e-04 | -3.169685e-03 | 5.622054e-03 | 5.622054e-03 |
Fracture0: +-----+-----+-----+-----+-----+-----+
Fracture0:
Finished Advancing MultiApps on TIMESTEP_END

```

Figure 3.6: Terminal output for FixedPointPostProcessors

In this figure, we can see that 1 Picard $|R| = 0$, which means that the Picard iteration has converged there but still we go one more step and realize that the reported reaction force changes in the second iteration. The phase field parameter doesn't change after the second iteration so there is now no need to go for the third iteration.

fracture.i

The fracture file is independent in itself in the sense that it receives some variable values from the parent file but all the MultiApp linking commands are there in the parent file.

The child (fracture) file starts with the mesh details code which should be the same as the parent. After this, we define the variables.

```

[Variables]
[d]
[]
[]

[AuxVariables]
[bounds_dummy]
[]
[psie_active]
  order = CONSTANT
  family = MONOMIAL
[]
[fy]
[]
[]

```

d that is the phase field parameter which was an auxiliary variable in the elasticity file, is a primary variable here. The active part of strain energy whose value at every quadrature point we get from the elasticity file and bounds_dummy are auxiliary variables.

```

[Bounds]
[irreversibility]
  type = VariableOldValueBoundsAux
  variable = bounds_dummy
  bounded_variable = d
  bound_type = lower
[]
[upper]
  type = ConstantBoundsAux
  variable = bounds_dummy
  bounded_variable = d
  bound_type = upper
  bound_value = 1
[]
[]

```

VariableOldValueBoundsAux is used to set the bound for the bounded variable to its old value. Bounds dummy is the name of the variable that this object applies to (it can't be applied directly to the phase field variable). Similarly, ConstantBoundsAux sets the bound to a constant which in our case is 1.

```

[Kernels]
[diff]
  type = ADPFFDiffusion
  variable = d
  fracture_toughness = Gc
  regularization_length = l
  normalization_constant = c0
[]
[source]
  type = ADPFFSource
  variable = d
  free_energy = psi
[]
[]

```

The equation that we want to solve in the fracture file is,

$$-\nabla \cdot \frac{2G_c l}{c_0} \nabla d + \frac{\partial}{\partial d} \left(\alpha \frac{G_c}{c_0 l} + \psi_0^+ \right) = 0 \quad (3.10)$$

The first term in this equation is considered the diffusion term and the second term as a whole is considered a source term.

```

[crack_geometric]
  type = CrackGeometricFunction
  f_name = alpha
  function = 'd^2'
  phase_field = d
[]
[psi]
  type = ADDerivativeParsedMaterial
  f_name = psi
  function = 'alpha*Gc/c0/l+g*psie_active'
  args = 'd psie_active'
  material_property_names = 'alpha(d) g(d) Gc c0 l'
  derivative_order = 1
[]
[]

```

Coefficients for the terms in the kernel are defined using the Material block where we can also define the crack geometric function known as $\alpha(\phi)$ in our literature.

```

[Executioner]
  type = Transient

  solve_type = NEWTON
  petsc_options_iname = '-pc_type -pc_factor_mat_solver_package -snes_type'
  petsc_options_value = 'lu          superlu_dist          vinewtonrsls'
  automatic_scaling = true

  nl_rel_tol = 1e-8
  nl_abs_tol = 1e-10

  dt = 0.001
  end_time = 0.1
[]

```

Finally, we have the executioner part of the fracture where different PETSc options are used for bounds to have an effect.

Final output of files

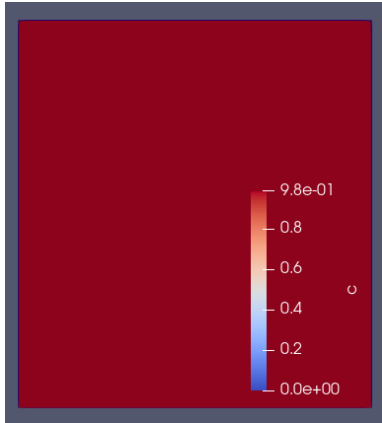


Figure 3.7: Final value of phase field parameter c

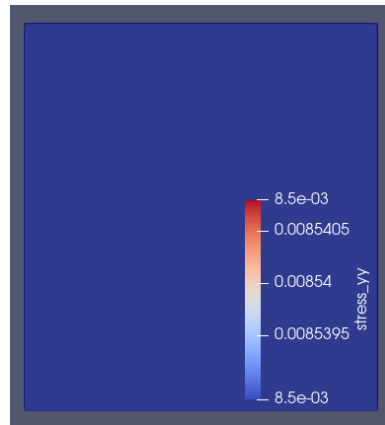


Figure 3.8: Final value of stress_yy

These are the elasticity.i output images after the last load step. We can confirm that the final values match and are constant throughout the element.

3.2.4 Comparison of results

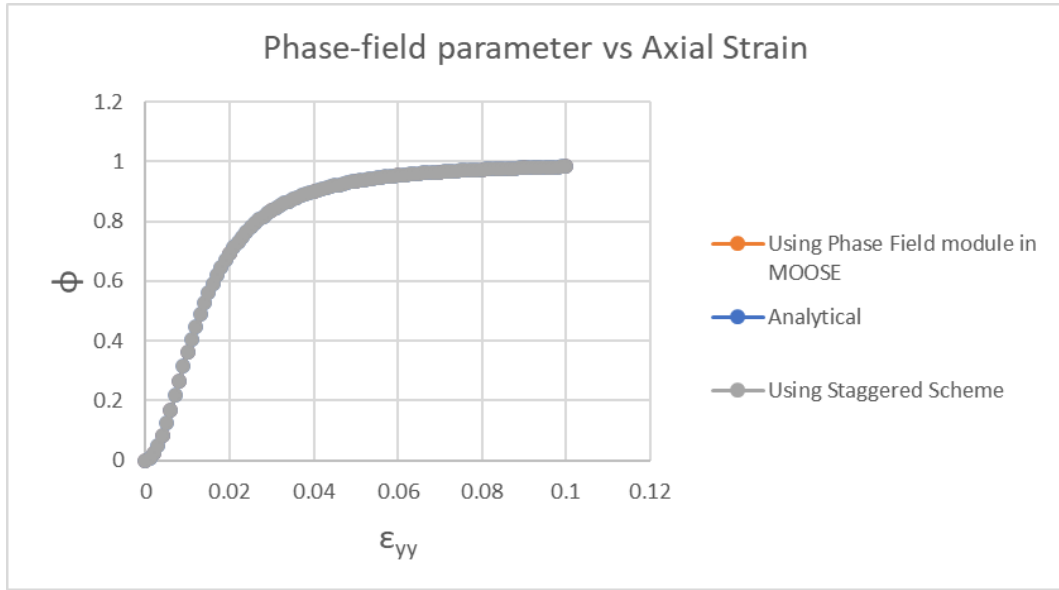


Figure 3.9: Phase-field parameter versus axial strain

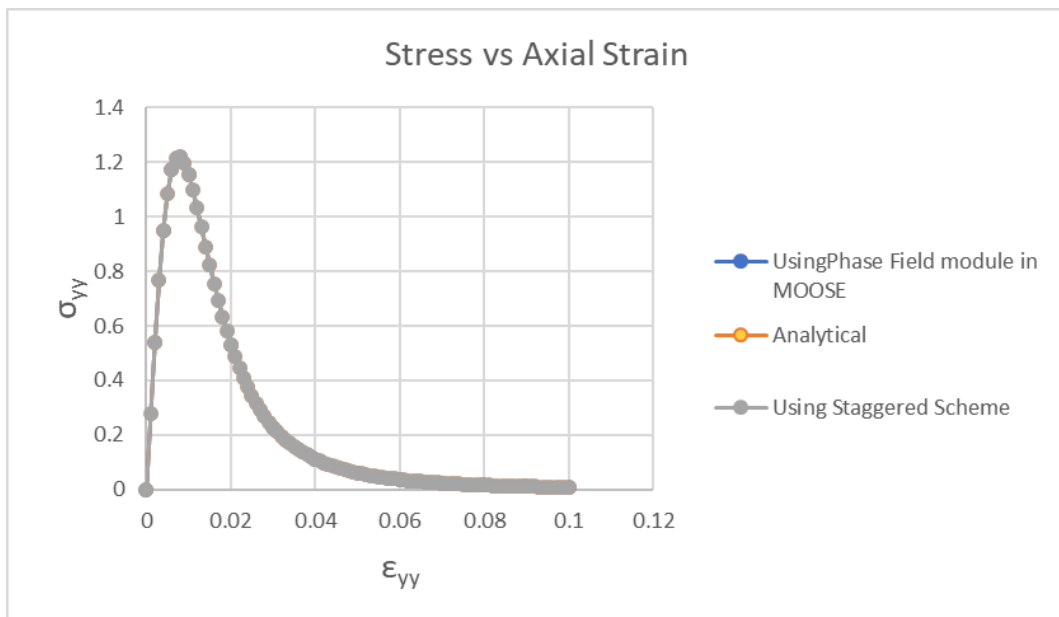
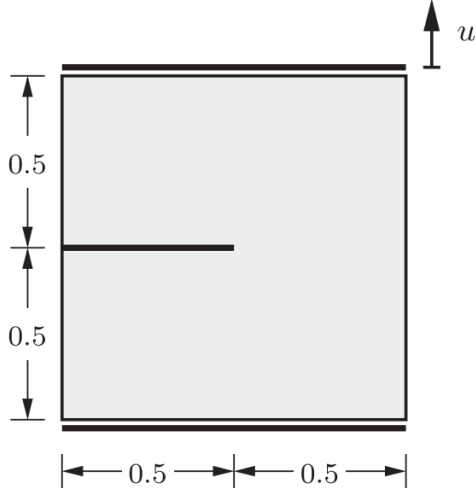


Figure 3.10: Stress versus axial strain

The obtained analytic solution and the output from MOOSE were plotted with the available data points and are showing remarkable accuracy. This is because of the linear nature of the problem.

3.3 Miehe Tension test [14]

In the previous section, we looked at cases where there was no crack or discontinuity present before loading. Now we will look at various other cases having different shapes and pre-existing cracks and discontinuities.



Property	Value
G_c	2.7×10^{-3} kN/mm
K	175 kN/mm ²
μ	80.679 kN/mm ²
l	0.0075 mm
δu	10^{-5} mm (linear)

Table 3.2: Material Properties

Figure 3.11: Single edge-notched Tension specimen

The single-edge notched Miehe tension test is used to study the behavior of materials under tensile loading with a pre-existing crack. It is a plane strain problem. Geometry and BCs can be referred from the figure. The bottom edge is fixed and displacement load is applied at the top. Material properties are also given in the table. We have used the same files as used for the first problem with appropriate changes to solve this problem using the staggered scheme.

3.3.1 Mesh and Discretization

To capture the crack propagation accurately, the mesh is refined in the expected crack propagation zone i.e. the centered strip of the specimen. This refinement results in a discretization with 20,000 elements and an effective element size of approximately 0.001 mm is required in the critical zone.

Mesh plays a crucial role in this problem especially when the specimen approaches the growth of the crack. We had many iterations on how the refinement should happen and what should be the size of the refinement zone as it is not very well defined in the literature.

How to model the crack in the mesh was also an area that required special attention. We are going to present all the different iterations that we used for meshing and later in the results section we will discuss how they affected the solution.

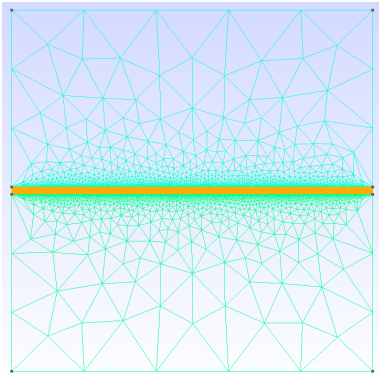


Figure 3.12: Mesh iteration 1

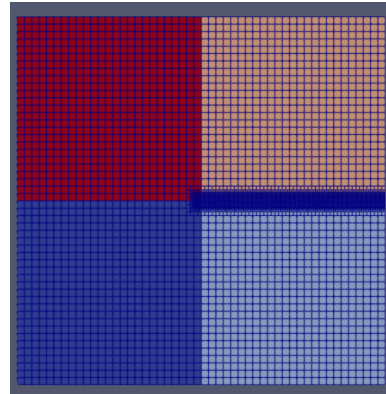


Figure 3.13: Mesh iteration 2

The basic requirement of mesh in the critical zone is that the element size should be at least smaller than $l_0/2$ (characteristic length). Following that as a first iteration we refined the whole center region using *gmsh* [8] and its transfinite properties where we can increase

the number of divisions on a particular edge. We realized that refinement is required only where the crack is expected to propagate so this time using MeshAdaptivity in MOOSE itself we increase the number of divisions to 4 times in the crack area.

In the first iteration, we defined the crack by marking a rectangular area of width 2l and length 0.5 from the left end. All the nodes in this area were set to the Dirichlet boundary condition of the phase field parameter equal to 1.

For the second iteration we created 4 parts of the original domain, defined pairing between the adjacent blocks leaving those two blocks that formed the interface of the crack. In this way, we were able to create a perfect slit where two surfaces split when load is applied.

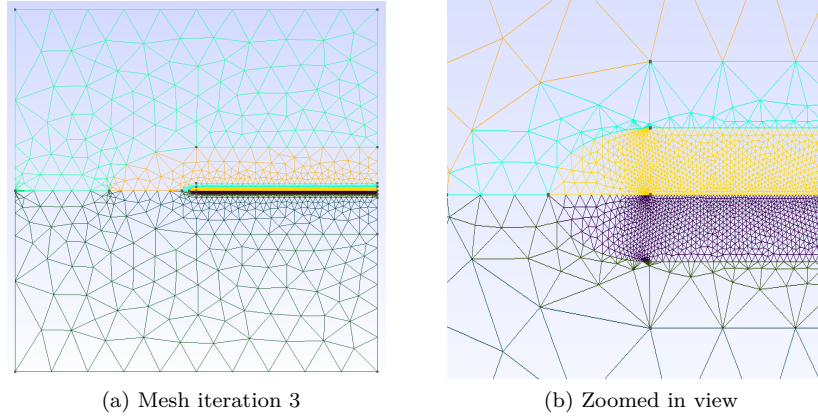


Figure 3.14: Representation of crack by a single line

Even after these two efforts we were getting a lot of unnecessary elements outside the crack zone. We referred to [17]. and found a new way of using elliptic zones. For the third iteration, we wanted a completely symmetrical mesh so we started by creating only the upper half of the mesh. Then using the Symmetry Duplicata feature of gmsh we mirrored all the elements for the lower half as well. Some things should be kept in mind while doing this operation.

1. We should be in the built-in mode of the gmsh
2. If we are in Cascade mode then we should also give the Coherence command so that there are no duplicate nodes on the edge about which we have symmetry
3. To copy the transfinite properties of the curves Geometry.CopyMeshingMethod should be set to 1
4. While saving mesh file from geo file, Mesh.SaveAll should be also set to 1 so that even if we have not defined some region/curve/point as a physical group we obtain mesh elements for those regions in our mesh file also

These commands mentioned above can be given from the user interface or by editing the .geo file.

Regarding the definition of the crack, this time we tried setting the value of the phase field parameter equal to 1 only along $x = 0.5$ and $y \leq 0.5$ but again we couldn't get the desired results. Referring to GitHub discussions we realize that at least two layers on both sides of the interface are required for the transition in the phase field values of the crack.

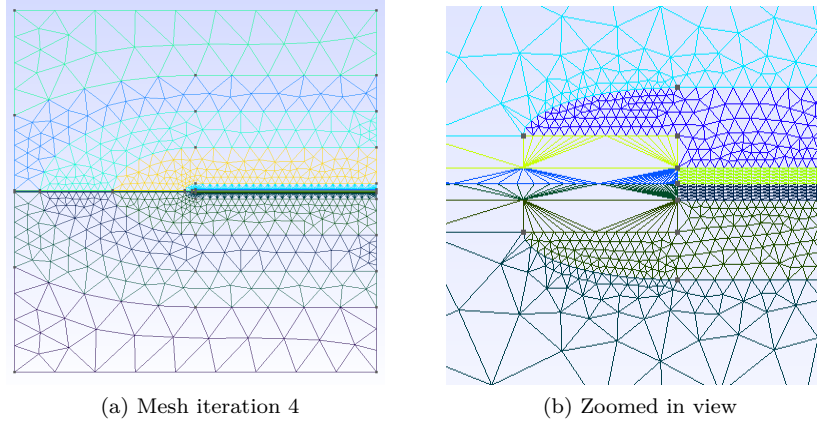


Figure 3.15: Representation of crack in 3 lines

Finally, we have arrived at this representation where for defining the crack we will use a very small width, and we will contain the transition of the phase field parameter to a combined width of $2l_0$.

3.3.2 Loading and Displacement

Initial displacement increments are set to $\delta u = 10^{-5} \text{ mm}$ for the first 500 loading steps. The displacement increment is adjusted to $\delta u = 10^{-6} \text{ mm}$ for subsequent loading steps to continue crack propagation. This increment again has a huge effect on the solution. We can use larger increments in the linear zone but if we continue that in the fracture or non-linear zone also no crack propagation is observed. We had different trials of the increment and its effect is reported in the results. In MOOSE we have the option of using TimeStepper functions through which we can change our dt at various times.

3.3.3 Solver and Tolerances

For the convergence criterion, we noted that in [14] Miehe has used the residual of the equilibrium equation. But what is the value of that residual tolerance and when the Picard iterations converge was also not well defined. So again we have to use different tolerances and note the nature of the results as obtained. For the linear part for which most of the results match we have used an absolute tolerance of 10^{-8} and a relative tolerance of 10^{-6} as the convergence criterion for both non-linear iterations within the app and Picard iterations in the MultiApp respectively.

Like using different time steppers we also tried to use different tolerances and solving methods like the hybre boomeramg mentioned in other MOOSE problems for different time intervals of the problem using the Restart_file_base command in MOOSE. We used different combinations in that as well and results are reported. How to implement the restart base is covered in the appendix.

3.3.4 Results

Crack growth obtained at different load points is reported and compared with Miehe results as shown below.

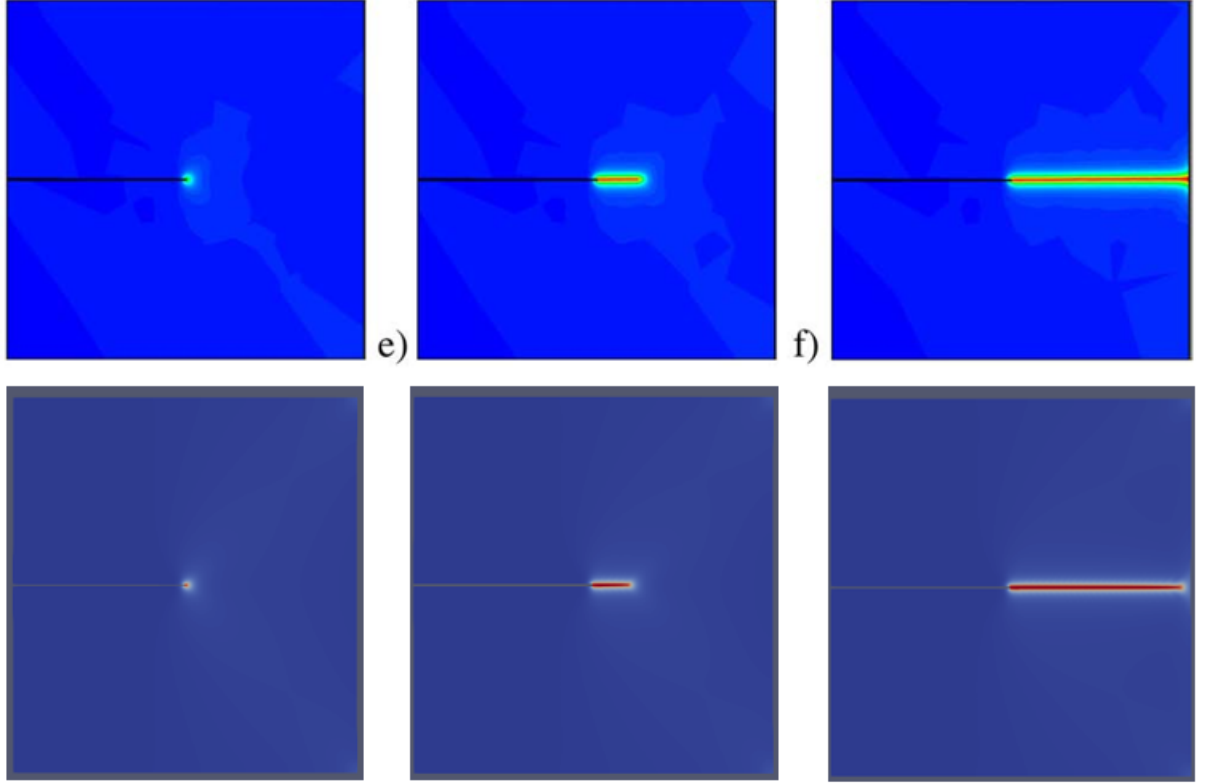


Figure 3.16: Crack pattern at different times

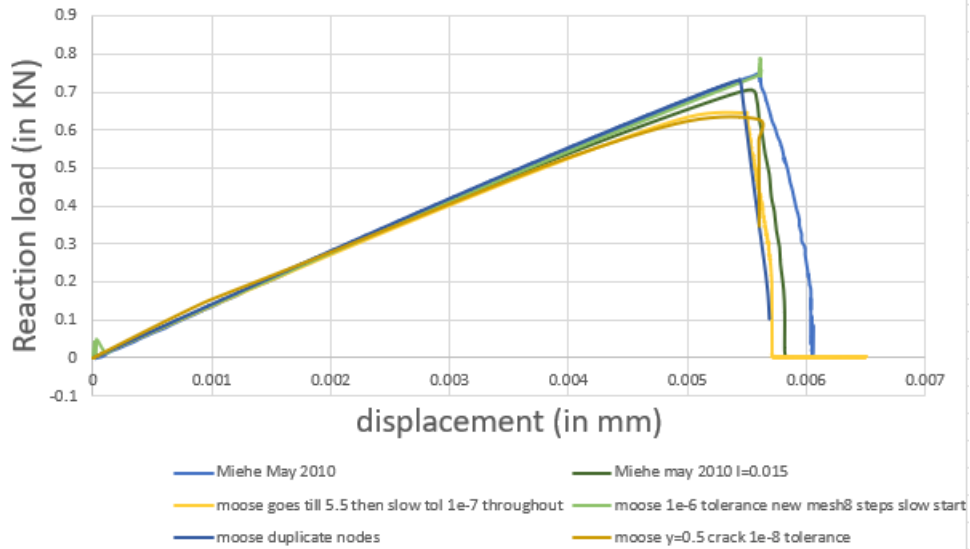


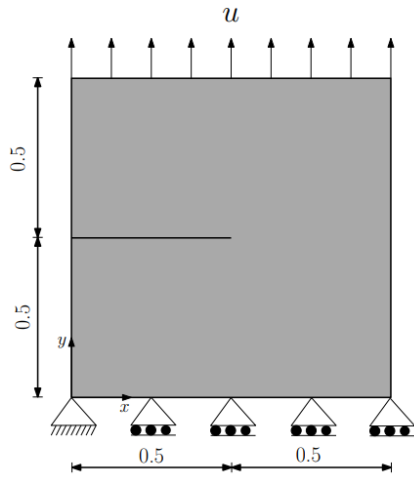
Figure 3.17: Load deflection curves for Miehe tension test

The crack growth is along the expected line of $y = 0.5$ only, but the time at which it is occurring is different from what is reported by Miehe. This is visible from load vs reaction plots as well where the linear part of the curves match for every simulation parameter but in the non-linear part we see a mismatch and the curve behaviour changes with each iteration, each response being different.

3.3.5 Discussion

The linear slope is coming same but in the non-linear zone mostly we see the crack propagation is earlier than expected. For less tolerances, we see some vibratory type of response at the start and near the start of crack propagation. We expected that with very small steps and very high tolerances, we would be able to achieve the results but that attempt was also unsuccessful as we faced the issue of solution not converging. We set the maximum number of iterations to 30 also but to no advantage. We found out the value kept oscillating because of the highly non-linear nature of the problem as for every Picard iteration it would present a different value. So in fact, less tolerances are recommended in the nonlinear zone with small time steps that give the best results.

3.4 Al_2O_3 and ZrO_2 Tension test [16]



Property	Al_2O_3	ZrO_2
E	380 kN/mm ²	210 kN/mm ²
ν	0.26	0.3
K_{IC}	5.2 MPa $\sqrt{\text{m}}$	9.6 MPa $\sqrt{\text{m}}$
l	0.03 mm	0.03 mm
δu	10^{-4} mm (linear)	10^{-4} mm (linear)

Table 3.3: Material properties

Figure 3.18: Geometry and Boundary conditions for Al_2O_3 and ZrO_2 Tension test

The tension test in this section is different from the previous problem in material properties and one boundary condition. In this problem, the complete bottom edge is not fixed in x-direction instead only one point is fixed. Modeling this test proves the phase field model's capability to model different material properties correctly.

3.4.1 Mesh and Discretization

The element size was chosen to be at least 5 times smaller than the characteristic length. It is to resolve the fracture zone for which authors have used nearly 1,60,000 nodes as they are using functionally graded material due to which they can't determine the crack direction beforehand. For, our analysis we have taken pure Al_2O_3 and pure ZrO_2 in two different files so we can use more number of nodes only along the centre strip like in the previous case.

For this example also we modeled our crack using both methods

1. Setting the value of the phase field parameter near the crack zone equal to 1:

For this point, it should be noted that it is a Dirichlet boundary condition for the phase field parameter, not an initial boundary condition. We have applied this condition through the auxiliary kernel setting the value of the auxiliary variable (d) to 1 only for the first step. In the fracture file, we have added it as a Dirichlet boundary condition to the primary variable (d) so now there's no need in the elasticity variable to again make its value 1.

2. Creating a sharp crack:

In the previous example we made four blocks and established no contact between one of the pairs but this time we were not able to use it because the top half containing elliptic curves can't be divided into two domains easily. Many surfaces we have to use if we want to achieve that.

In MOOSE there's one more way by which we can make sharp cracks. We make two halves and then establish a stitch between them along the part of the line where we want to establish the connection. The remaining line will behave like an interface because of no stitch between halves. In this, we should also ensure that the boundary IDs of the halves are different. An offset option is provided in MOOSE if we are making our mesh from it. For gmsh, in the .geo file, we have to manually set the ids of the physical surfaces to be different.

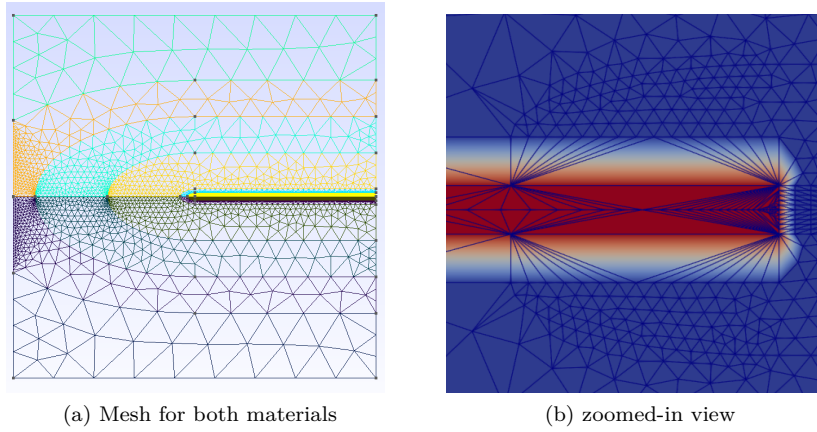


Figure 3.19: Understanding crack initialization

3.4.2 Loading and Displacement

Initial displacement increments are set to $\delta u = 10^{-4} \text{ mm}$ for the linear part of the load-deflection curves. For subsequent loading steps, the displacement increment is progressively adjusted to $\delta u = 10^{-5}$ and 10^{-6} mm to continue crack propagation.

3.4.3 Results

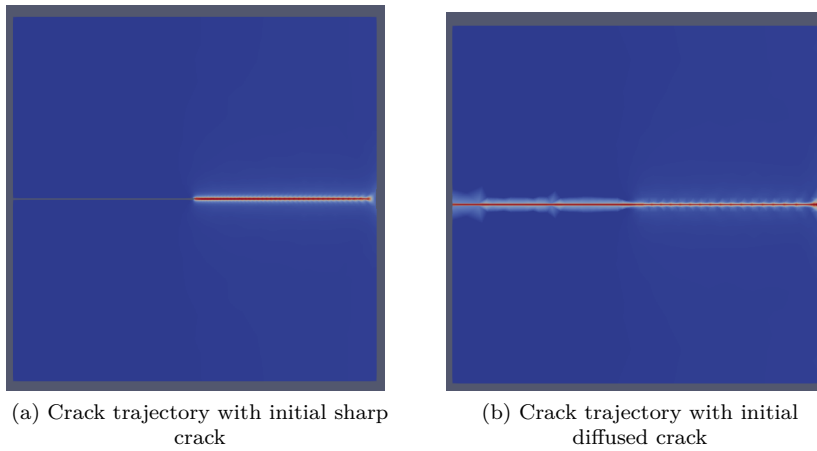


Figure 3.20: Comparison of crack growth for different definitions of initial crack

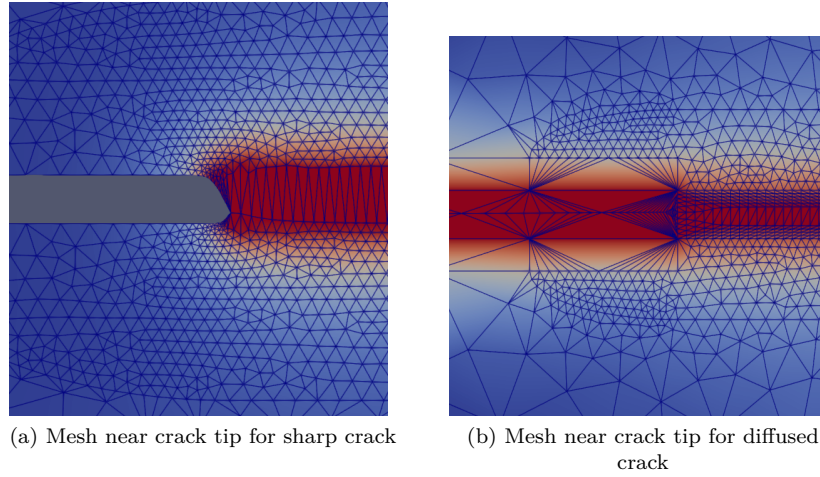


Figure 3.21: Comparison of mesh near crack tip for different definitions of initial crack

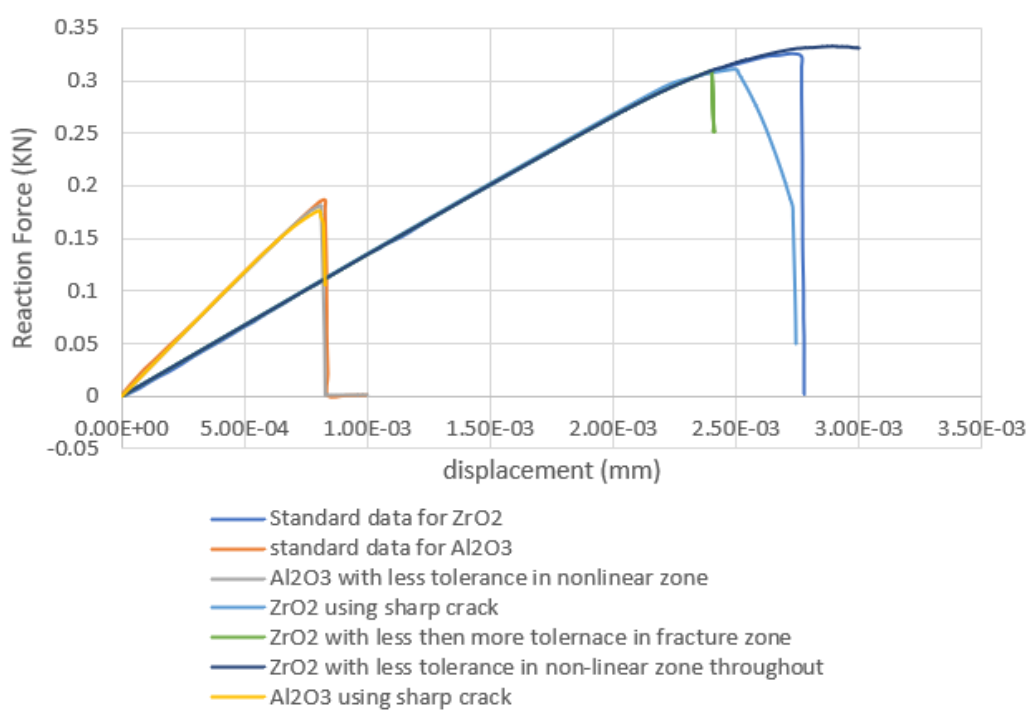


Figure 3.22: Load-deflection curve for Al_2O_3 and ZrO_2 combined

The crack growth patterns are similar for both materials with different maximum reaction forces and displacement loads for crack initiation as expected. Simulation results perfectly match for linear part but for non-linear different combinations give different results as for the Miehe tension test.

One phenomenon that we observed quite often is that whenever we transform to a tighter tolerance or to a smaller time near the non-linear zone, the curve doesn't grow itself to the point where crack initiation is expected. Instead, the crack initiation happens at that transition point only, propagates slowly, and then fails rapidly toward the end.

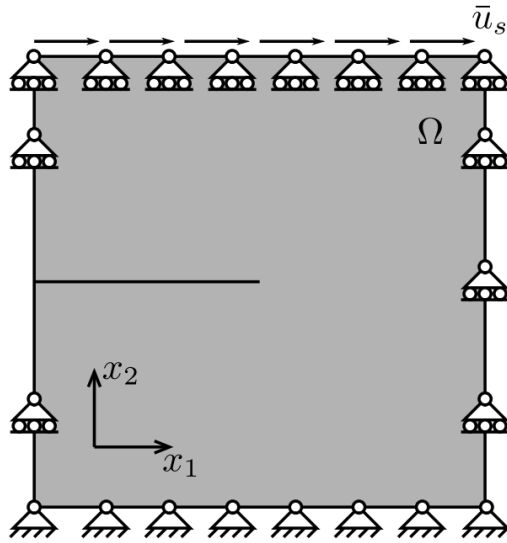
3.4.4 Discussion

This time we have presented results where we changed the tolerances for the non-linear part also. From those results and other iterations, we have learned that most of the time the solid mechanics files take more iterations to converge than the fracture files. In MOOSE,

when individual files fail to converge to their non-linear tolerances (for our cases maximum of 50 non-linear iterations are allowed), the solution is declared diverging and the simulation is stopped. However, as we explained above when the MultiApp or the parent file fails to converge to the fixed point tolerances for which a maximum of 20 iterations are defined, the solution is still accepted with a message of maximum iterations reached, and the simulation proceeds to the next step.

This also has a very important bearing on the results as the number of fixed-point iterations is equal to the number of times equations are solved using the staggered scheme. In the crack propagation zone, more staggered iterations lead to sharper crack descent. In some cases, however, we have found the same result with 10 and 20-fixed-point iterations. So, it is important to decide what is the maximum number of iterations we want to keep the simulation time minimum while maintaining accuracy in the results.

3.5 Miehe Shear test [14]



Property	Value
Gc	$2.7 \times 10^{-3} \text{ kN/mm}$
λ	121.15 kN/mm^2
μ	80.77 kN/mm^2
l	0.0075 mm
δu	10^{-5} mm

Table 3.4: Material properties for shear test

Figure 3.23: Geometric and Boundary conditions for Miehe Shear Test

The single-edge notched pure shear test involves a similar squared plate with a horizontal notch subjected to shear loading instead of tension. The goal is to investigate the material response under shear deformation. Material properties are also the same as for the Miehe Tension test.

3.5.1 Mesh and Discretization

The mesh is refined in the expected crack propagation zone, which is the lower right diagonal for this problem to capture the crack propagation accurately. The refinement results in a discretization with 30,000 triangular elements and an effective element size of approximately 0.002 mm is used when the crack is modeled with the phase field parameter defined as 1 near the crack.

For the second case when we modeled the crack by a sharp line we used 16000 square elements. We used the stitch method discussed earlier to make the slit. For the refined zone part we used the Adaptivity feature in the moose and refined the elements in the desired area after marking them using OrientedBoxMarker

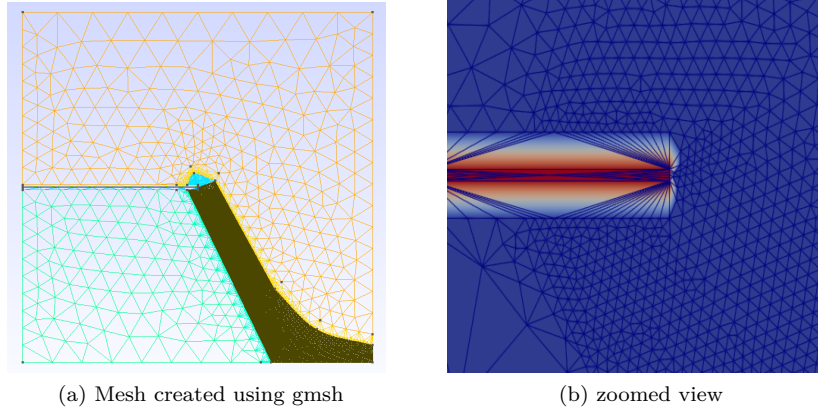


Figure 3.24: Mesh for representing crack through the phase-field parameter

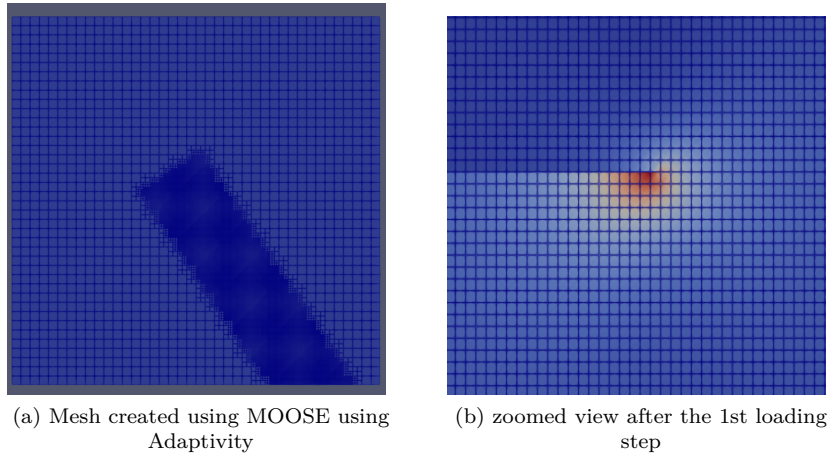


Figure 3.25: Mesh for representing sharp crack

3.5.2 Loading and Displacement

Initial displacement increments are set to $\delta u = 10^{-5} \text{ mm}$ for the first 500 loading steps. For subsequent loading steps, the displacement increment is adjusted to $\delta u = 10^{-6} \text{ mm}$ due to the brutal nature of crack propagation. For our simulation, we used $\delta u = 1.5 \times 10^{-5} \text{ mm}$ throughout the run taking reference from a standard problem given in [7] and it produced matching results.

3.5.3 Results

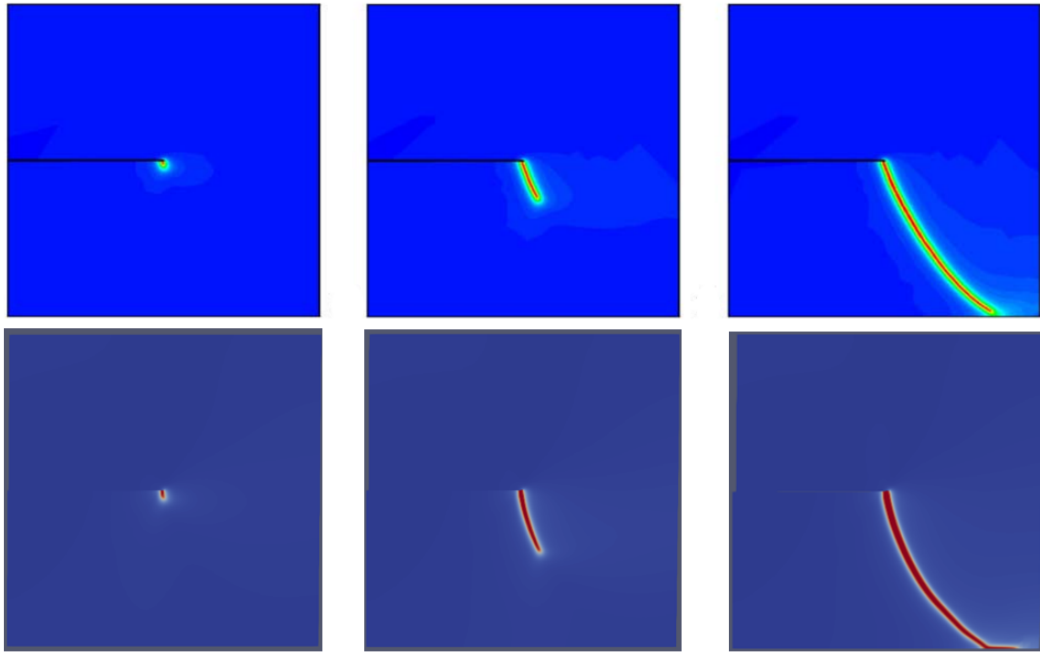


Figure 3.26: Single edge notched pure shear test crack pattern for different displacement load

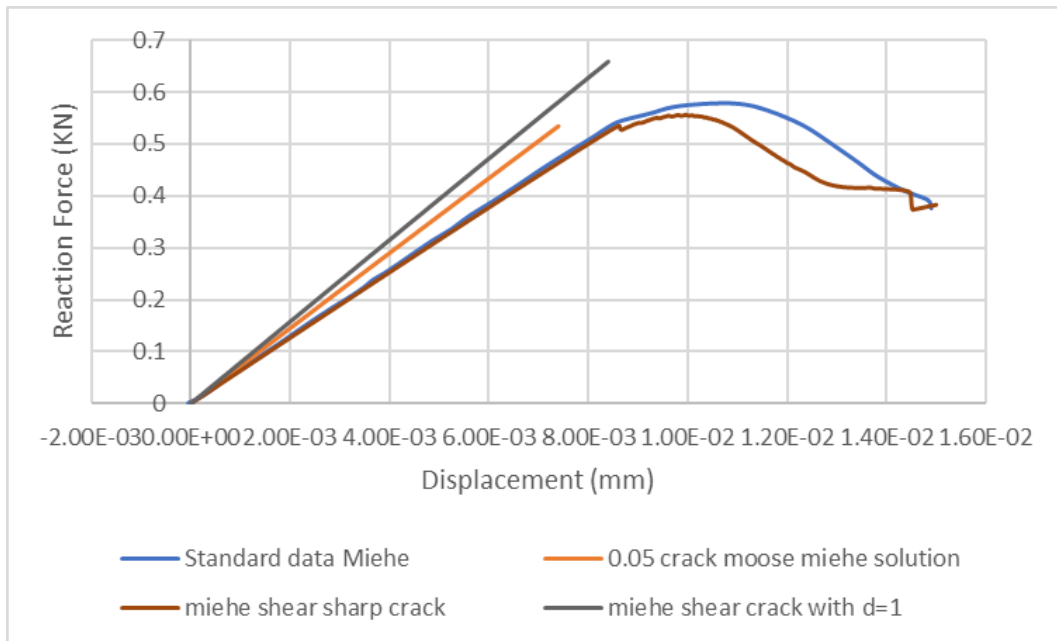


Figure 3.27: Load-deflection curve for Single edge notched pure shear test

The results match to an extent for the sharp crack except for the later part of the crack propagation but for the diffused crack, we are getting more slope than expected. Also, after refining the pre-crack zone or modeling it with less thickness, the slope got further increased.

3.5.4 Discussion

Knowing the shape of the expected crack propagation zone was important for this problem. Also, the time steps need to be small enough otherwise the crack will not initiate. The diffused crack problem also faced the problem of the crack not initiating even after the load-deflection curve crossed its maximum load value suggested by standard results. The tolerances used for this problem were 10^{-6} for the diffused problem and 10^{-8} for the sharp case.

3.6 Symmetric 3-point bending test [14]

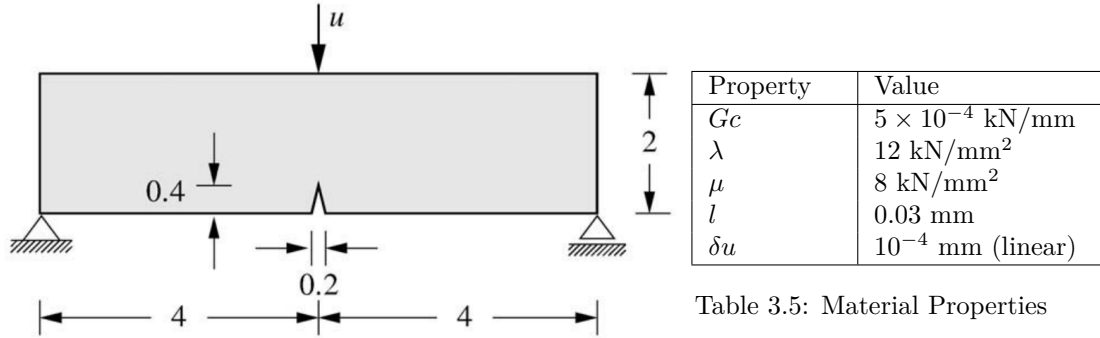


Table 3.5: Material Properties

Figure 3.28: Geometry and Boundary conditions

The symmetric three-point bending test is a classical benchmark problem often analyzed for its significance in the study of fracture mechanics. The test involves a simply supported notched beam subjected to a bending load at the midpoint and contains a central notch to initiate crack propagation.

3.6.1 Mesh and Discretization

To capture the crack propagation accurately, the mesh is refined in the expected crack propagation zone. This refinement results in a discretization with 20,000 elements and an effective element size of approximately 0.0008 mm is required in the critical zone.

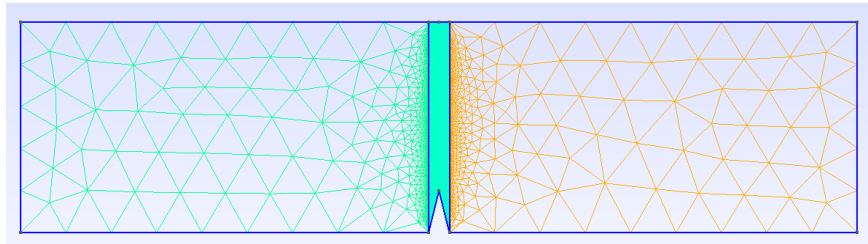


Figure 3.29: Refined mesh near crack propagation zone

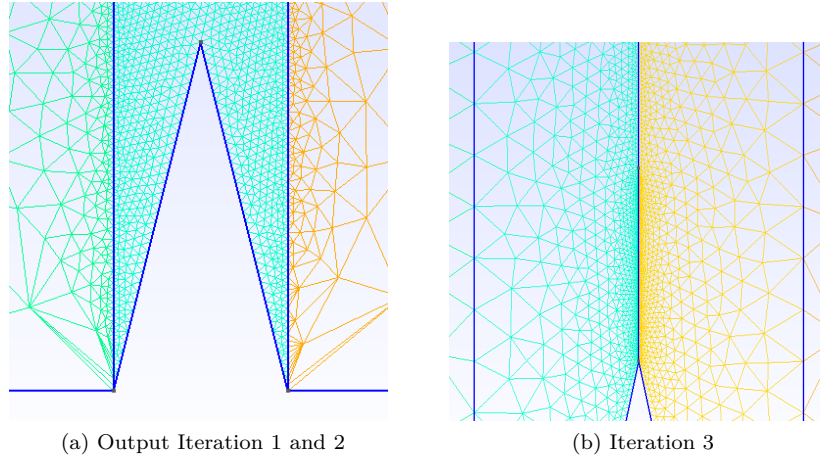


Figure 3.30: Representation of mesh near the notch with different approaches

Here also we had 3 iterations of mesh. For 1st iteration, we gave geometric progression to the elements to become very small near the tip. The total elements were little more than 20,000. In the second iteration, we had no progressions so it decreased the number of elements to 19,000. In the third iteration, we made a center line about which we were finally able to achieve the size of 0.0008 mm with 20,000 elements only.

3.6.2 Loading and Displacement

Initial displacement increments are set to $\delta u = 10^{-3} \text{ mm}$ for the first 360 loading steps. For subsequent loading steps, the displacement increment is adjusted to $\delta u = 10^{-6} \text{ mm}$ to continue crack propagation.

3.6.3 Results

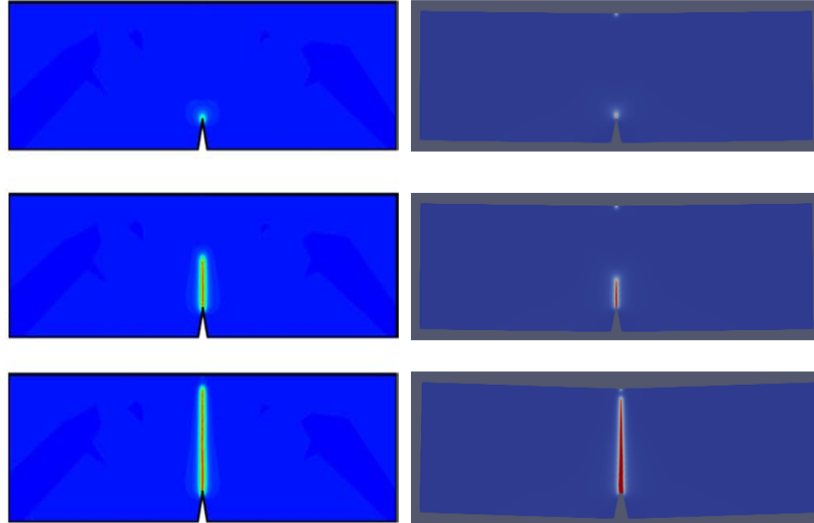


Figure 3.31: Crack pattern for three-point bending test

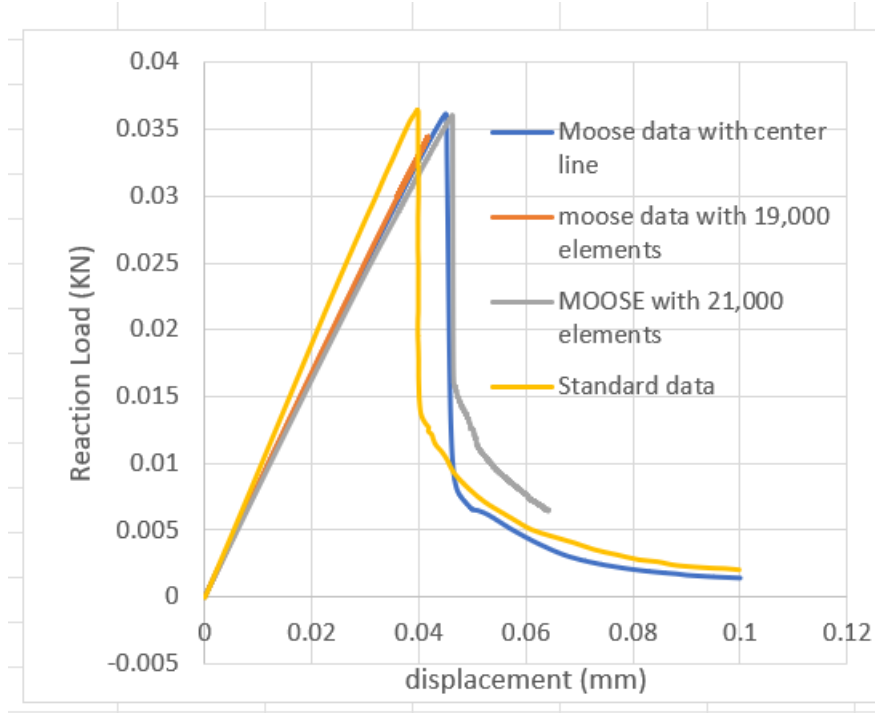


Figure 3.32: Load-deflection curves for different meshes

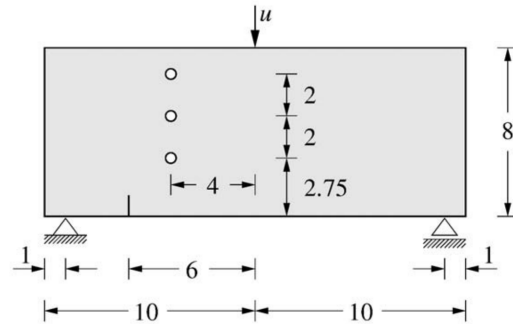
Similar to the tension test we have got our crack growth in the expected region and along expected lines but the timing is different. Contrary to the tension case this time the crack initiation starts at a later time however the maximum force at which it occurs is very close.

3.6.4 Discussion

Linear slope is different but the shapes are similar with an offset. We were able to achieve the element size of 0.0008 mm in the critical zone still the linear slope doesn't change significantly, however in that case non-linear matches very closely.

In the literature the loading rate given is very slow, we had fast iterations also but no significant change we observed. The absolute tolerance for this problem we kept to 1e-8 only.

3.7 Asymmetric notched beam test [14]



Property	Value
Gc	1×10^{-3} kN/mm
λ	12 kN/mm ²
μ	8 kN/mm ²
l	0.025 mm
δu	10^{-3} mm (linear)

Table 3.6: Material properties for asymmetric notched beam test

Figure 3.33: Geometry and Boundary conditions for Asymmetric notched beam test

The asymmetric notched beam used in the three-point bending test features three holes. The geometry, loading, and boundary conditions are detailed in the above figure and are

originally based on the work of Bittencourt et al. [4]

3.7.1 Mesh and Discretization

The mesh consists of 58,000 triangular elements, refined in areas where the crack is expected to propagate. The refinement was not exactly defined and we created it using a spline in gmsh made of points around the crack trajectory obtained via online plotting software.

The position of the notch is very important for this configuration along with the width. In our mesh, we have given a width of 2l and tried both rectangular and circular ends but the result obtained was the same.

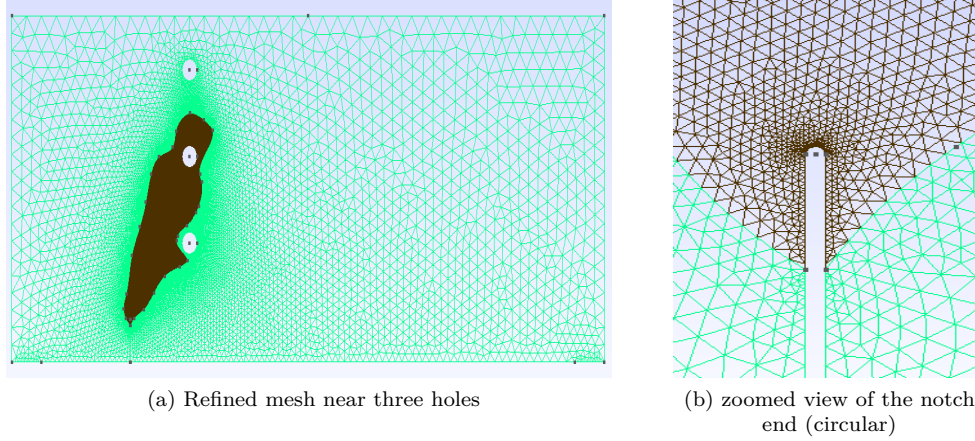


Figure 3.34: Representation of mesh for asymmetric notched beam test

3.7.2 Loading and Displacement

Initial displacement increments are set to $\delta u = 10^{-3} \text{ mm}$ for the first 200 loading steps. For subsequent loading steps, the displacement increment is adjusted to $\delta u = 10^{-4} \text{ mm}$. This information we have gathered from Ambati et al. [1] along with $l = 0.01 \text{ mm}$ to match their results as well.

3.7.3 Results

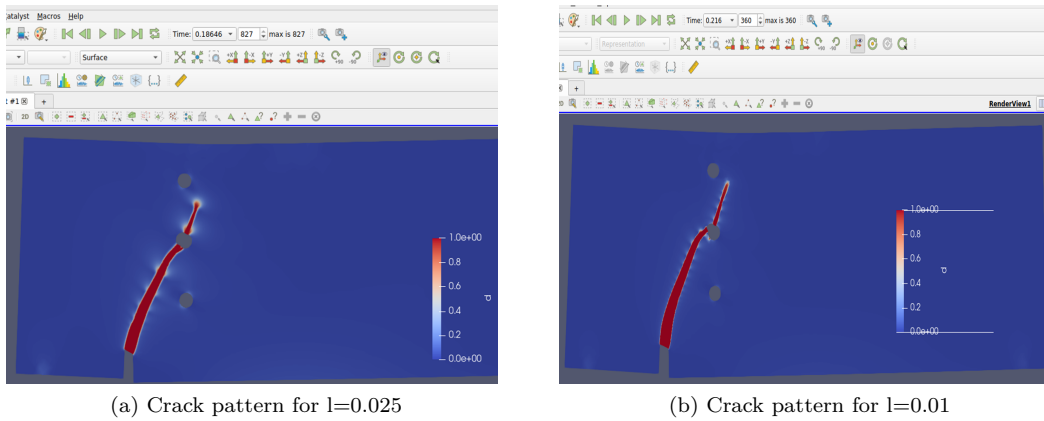


Figure 3.35: Final crack pattern for asymmetric notched beam test

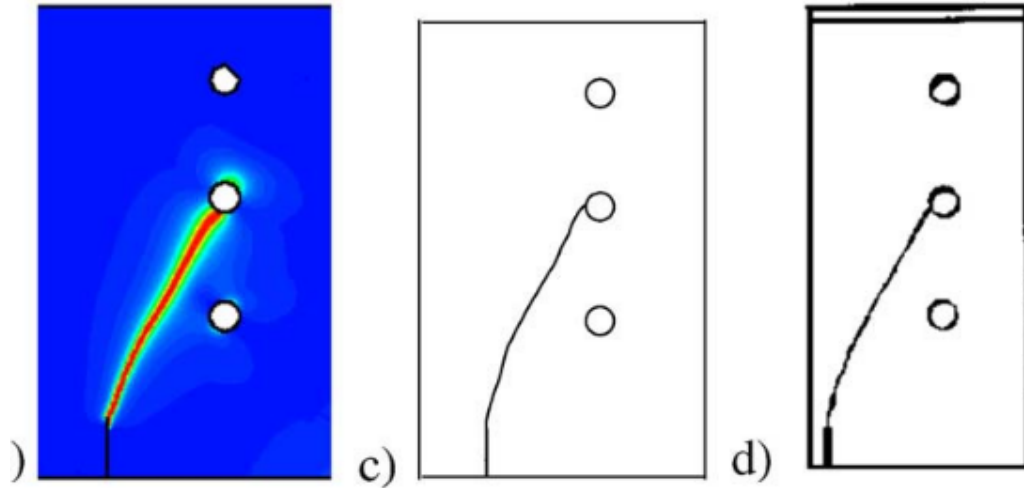


Figure 3.36: Standard results i) Miehe et al. [14] ii) Miehe and Gueres [13] iii) Bittencourt et al. [4]

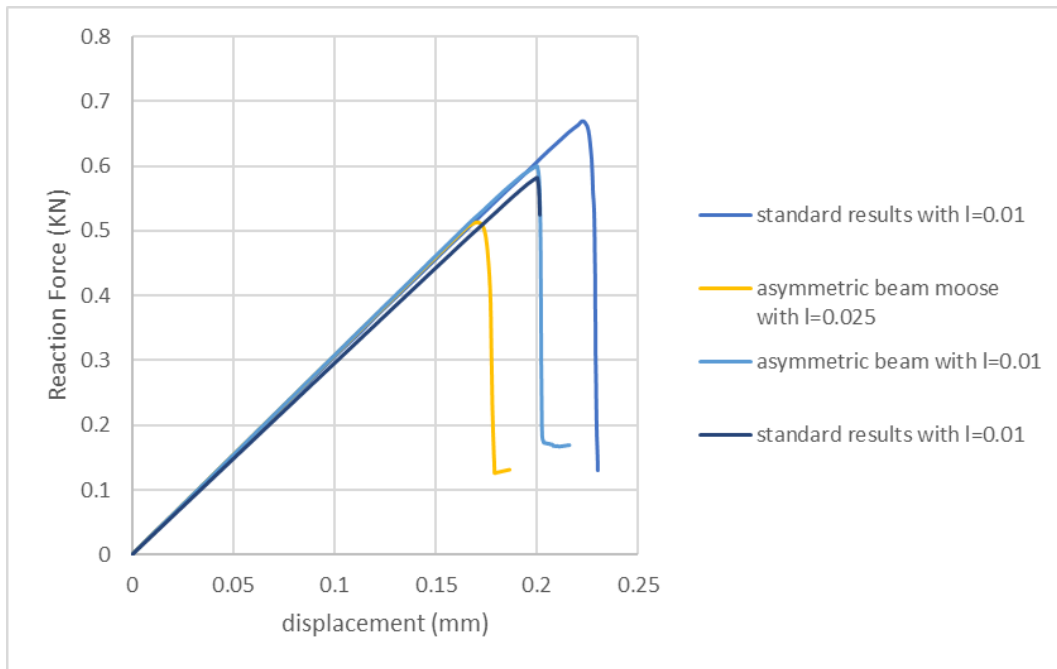


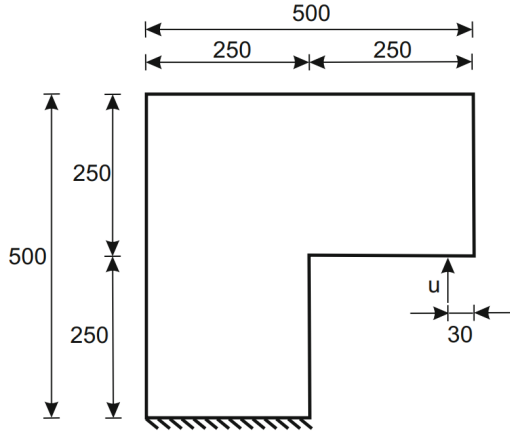
Figure 3.37: Load-displacement curves for the asymmetrically notched beam with standard result taken from Ambati et al. [1]

The crack trajectory seems to follow the path as suggested in the literature but like the previous results the timing doesn't match and because we took a large end time we have shown the crack trajectory grown beyond what has been reported in the literature.

3.7.4 Discussion

For this result, like the tension test, the linear region perfectly matched but the crack initiated earlier than expected. We also learned that a change in the characteristic length has a drastic effect on the crack initiation but no effect in the linear region when stresses are low. Solvers and tolerances could also have a role in this problem but we have not explored that.

3.8 L-shaped panel test [1]



Property	Value
Gc	$8.9 \times 10^{-5} \text{ kN/mm}$
λ	6.16 kN/mm^2
μ	10.95 kN/mm^2
l	1.1875 mm
δu	10^{-3} mm

Table 3.7: Material properties for L-shaped panel test

Figure 3.38: Geometry and Boundary conditions for L-shaped panel test

This test involves an L-shaped panel subjected to a bending load at a distance of 30 mm from the right side and no initial pre-crack is prescribed. We apply a cyclic load in this problem so it tests our model against crack-closure which is a necessary aspect to check.

3.8.1 Mesh and Discretization

Discretization with 9650 quadrilateral elements is prescribed for this problem but because MOOSE can either handle a 3-noded or a 4-noded element one at a time, we have gone ahead with 28,172 triangular elements with local refinement in the areas where the crack is expected to grow. For the local refinement, we had to make 2 iterations of mesh. In the 1st case, the crack grew outside our estimated refined zone with an upper boundary of 275 mm so for mesh 2 we increased it to 300 mm .

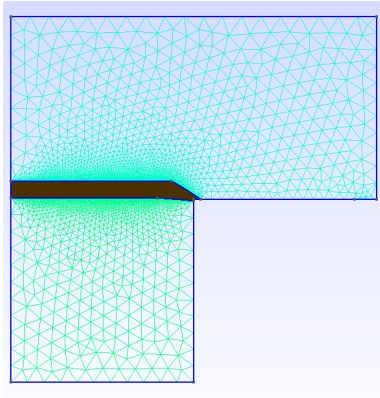


Figure 3.39: Mesh refinement for L-shaped panel

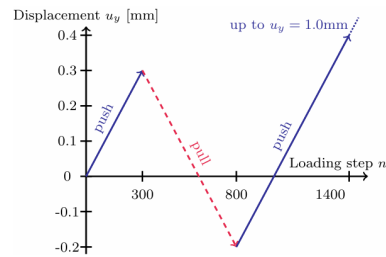


Figure 3.40: Loading rate on L-shaped panel

3.8.2 Loading and Displacement

The displacement loading is cyclic with a fixed increment of $\delta u = 10^{-3} \text{ mm}$.

3.8.3 Results

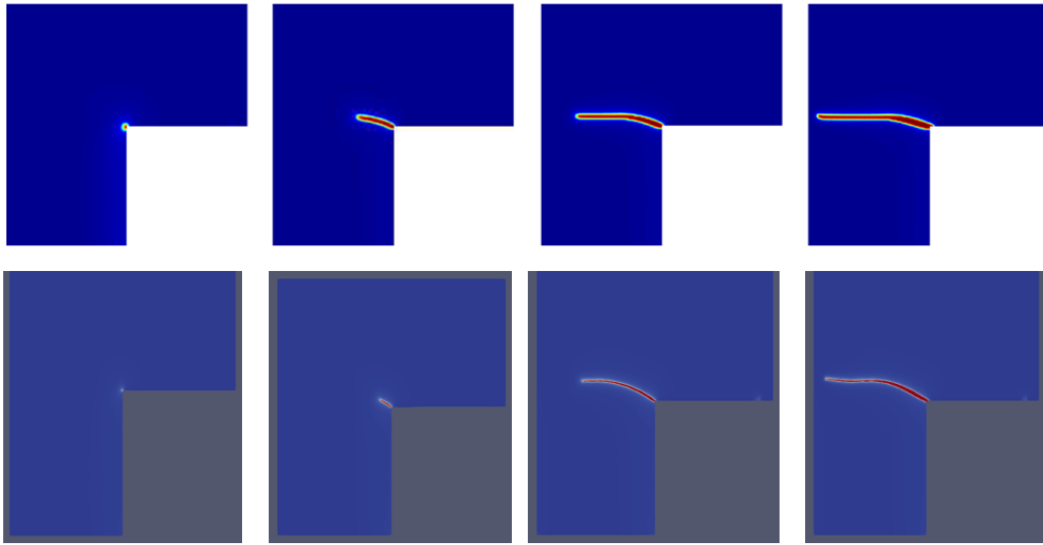


Figure 3.41: Crack growth at displacements i) 0.22 mm ii) 0.3 mm iii) 0.45 mm and iv) 1 mm

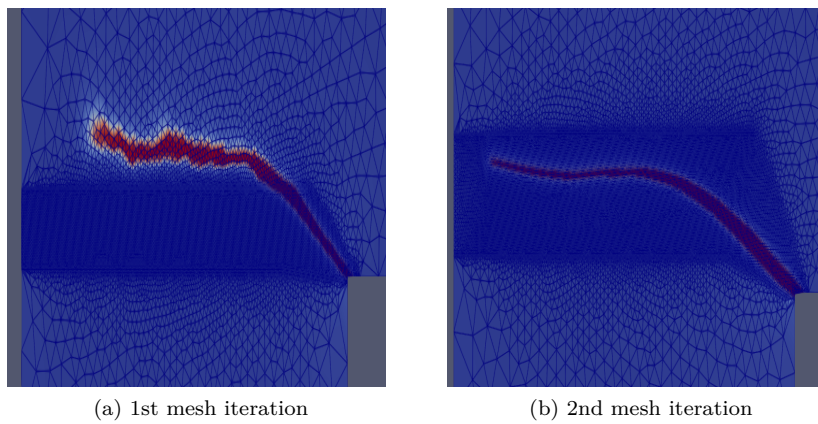


Figure 3.42: Final crack growth for two different refinement zones

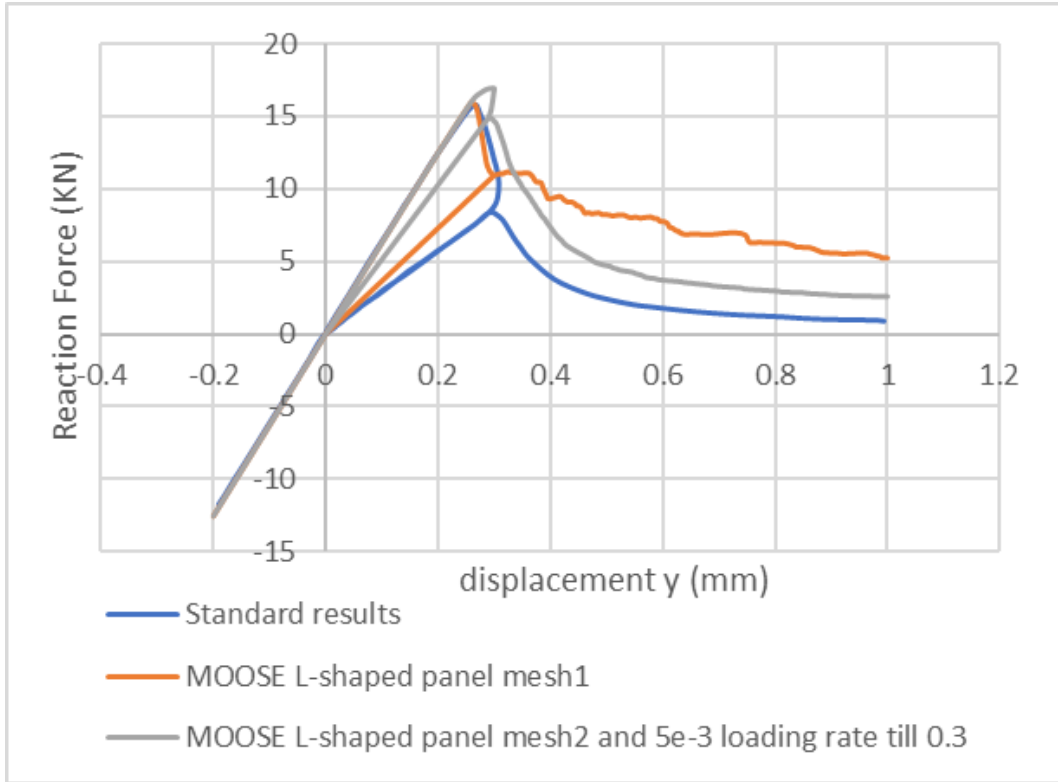


Figure 3.43: Load vs deflection curve for L-shaped panel test

We have an exact match in the curves for linear regions both while pushing and pulling. Crack initiation also took place this time almost at the same time and load but the crack growth was not the same due to which we get different curves than expected.

For iteration 2 of mesh we expected a similar response to the 1st iteration due to which we increased the loading rate to $5e-3$ mm for 0 to 0.3 mm, however, that attempt increased the time of crack initiation. Later with mesh 2, we got better results in the non-linear zone.

3.8.4 Discussion

So, the phase field model and codes have worked for us well in ensuring that the crack closure or decrease in the phase field value of the crack region doesn't happen while the pulling load is applied. Tolerances and solvers for this problem also are not varied to check for better results.

In a note for all the problems above, the viscosity parameter which many authors have included in their work, we have not included it in our formulation and it is considered zero everywhere.

Chapter 4

Conclusion

This report made an effort to thoroughly review the phase-field literature and validate the most commonly used benchmark problems. Building on our prior knowledge, we started with Griffith's energy balance equation. I have tried to emphasize time and again that the phase-field parameter just interprets the crack differently. It converts the crack to a functional quantity, the minimization of which gives us the path of the crack based upon the minimization of energy which is a very fundamental concept. We went through all the terms that come up due to this definition and how each has a role to play and ultimately affects ϕ or σ . After the phase field, we worked our way towards MOOSE. Having no background in the software, we again started from what we already knew, i.e., heat-transfer and Tensor Mechanics problems. After that, we solved the uniaxial plane strain problem using the phase field module and staggered scheme using the Multiapp feature in MOOSE and exactly matched the results. For other benchmark problems, we had to delve deep into many parameters especially meshing and how to model the crack, how to use the solver, etc. In this process, we learned about many features of gmsh also. After the quasi-static problems we attempted to solve some dynamic problems but for that purpose the adaptability of the RACCOON app for dynamic problems could not be ascertained due to which we could not proceed further within the limited time that we had. We might not have been even able to exactly match the results as we did for the first problem, but it has given us a good understanding of all the associated parameters of the phase field which would serve as a learning for validating more benchmark problems in the future.

Future work

1. Validation of all benchmark problems in Phase-field literature through the MOOSE framework and extending it to solve impact problems in layered materials and showcasing the crack trajectories
2. Study of dynamic crack problems

Appendix

A1: Heat transfer Module

The general 3D heat conduction equation is given by,

$$\nabla k \nabla T + \dot{q} = \rho c \frac{\partial T}{\partial t} \quad (4.1)$$

where, T is temperature, t =time, k =thermal conductivity, $\dot{q}$ =volumetric heating/source term, ρ =density and c =specific heat capacity

If we look at any general problem of heat conduction, along with this governing equation, it will have a system/body defined through geometry on which the analysis will be done. We will have Boundary/Initial conditions based on the type of problem and unknowns we need to find, and we need to supply material properties that are given. MOOSE also accepts an input file in a similar format where we need to define the problem completely with the solver that we want to use.

It is to be kept in mind that the governing equations that will be used in this analysis will be already defined in the modules along with their finite element formulation/weak form. We may have a look at it to get a correct idea of solutions and the quantities that we need to provide, or we can alter them if required, but such changes are suggested to be done in our own application so that we can also have a record of the changes we made in our GitHub repository. This will be explained more in the application development section. For now, let's look at some heat conduction examples which will help us to understand the input files better.

1-D heat conduction with volumetric heating

Consider a rectangular geometry having $T = 0$ at the left boundary and $T = 1$ at the right. Consider top and bottom boundaries having zero heat flux conditions so that we have a 1D problem. The volumetric heat generation and the thermal conductivity are also equal to 1. Units are consistent.

The analytical solution for this problem will be given by

$$T(x) = -\frac{x^2}{2} + 1.5x \quad (4.2)$$

This solution is obtained using the equation 4.1

Now, let's look at the MOOSE input file which is required to solve this problem block-by-block

Mesh

There are two ways by which we can create a mesh in MOOSE. First is by using the *GeneratedMesh* generator type which is useful for creating simple geometries and defining regular meshes. The second way is to import a mesh created in some other software such as Gmsh by entering *file = "filename.msh" or "filename.e"* inside the mesh block.

Here we have used the first method. We wanted to make a rectangular geometry hence the dimension of the mesh= 2. *nx* and *ny* specify the number of divisions we want along the x and y-axis, and *xmax* and *ymax* specify the maximum extent of the geometry which is set as 1. We also have the option of setting *xmin* and *ymin* under this type but that is

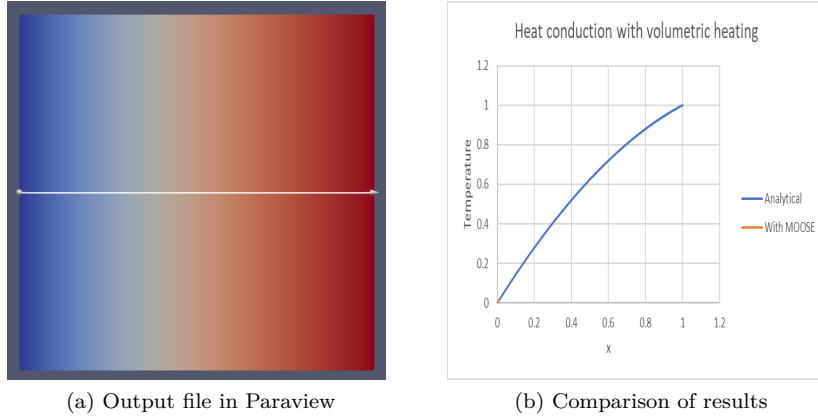


Figure 4.1: Representation of temperature profile taken along $y=0.5$

```
[Mesh]
  type=GeneratedMesh
  dim=2
  nx=10
  ny=10
  xmax=1
  ymax=1
[]
```

Figure 4.2: Mesh block of the input file

set to 0 by default so we have not changed that. The coordinate system definition is also cartesian by default.

One thing that is to be noted here is the requirement of more than 1 element for this problem. When we take one element, the effect of the volumetric heating on the temperature of the body inside its edges could not be taken into account as we have no nodes to carry that information. So we have taken 100 elements here so that the result is accurate and captures all variations.

Variables

```
[Variables]
  [Temperature]
  []
[]
```

Figure 4.3: Variable block

This is the section where you mention the variable that you want MOOSE to solve. Here we have called it *Temperature* but it can be given any name. We just need to be consistent in mentioning it wherever required. By default, MOOSE will assign variables to linear shape functions of the Lagrange family. We can change it to Hermite, Monomial shape functions of 2nd or higher order.

Kernels

This is the section where you pass your variable to the governing weak-form equations.

Materials

Used for defining all the material properties attached to the problem. One of the most important blocks, as it can be used to account for any additional parameter that has attached itself to the base variable and appears as a constant or variable in the weak form.

```
[Kernels]
  [temp4]
    type = HeatConduction
    variable = Temperature
  []
  [heat_source]
    type = HeatSource
    variable = Temperature
    value = 1
  []
[]
```

Figure 4.4: Kernel block

```
[Materials]
  [thermal]
    type = HeatConductionMaterial
    thermal_conductivity = 1
  []
[]
```

Figure 4.5: Material block

BCs

```
[BCs]
  [top]
    type = ADDirichletBC
    variable = Temperature
    boundary = right
    value = 1
  []
  [bottom]
    type = ADDirichletBC
    variable = Temperature
    boundary = left
    value = 0
  []
[]
```

Figure 4.6: BCs block

In this section, the boundary conditions given with the problem are reported. Majorly we have two types of BCs: Dirichlet and Neumann. The AD which is visible here refers to the automatic differentiation which is due to the solver type. One thing to be noted here is that Neumann conditions are set by default on those boundaries where no condition is mentioned. That's why for 1D analysis here, we mentioned only two boundary conditions in the input file.

Executioner

```
[Executioner]
  type = Steady
  solve_type = NEWTON
  petsc_options_iname = '-pc_type -pc_hypre_type'
  petsc_options_value = 'hypre boomeramg'
[]
```

Figure 4.7: Executioner block

This part is concerned with ways to solve the final matrix of the problem relating to the knowledge of the Jacobians. For transient problems, the type will be changed to Transient. For more details, we can refer to MOOSE documentation.

```
[Outputs]
  exodus = true
  [csv]
    type = CSV
  []
[]
```

Figure 4.8: Output block

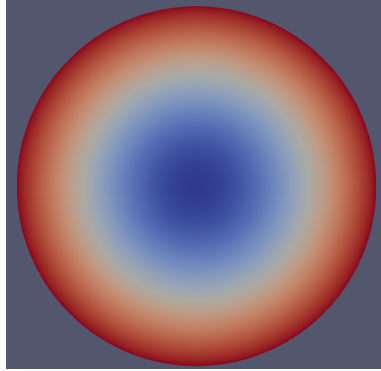
Outputs

By setting Exodus= true, we demand MOOSE to write an Exodus II mesh and solution file to the disk which can be interpreted by Software like ParaView and Peacock. We can also request .csv output here which can be read in Excel.

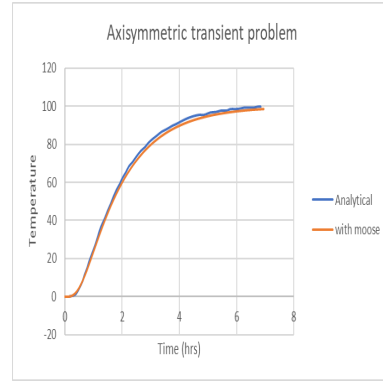
Axi-symmetric Transient heat conduction

Now we consider a sphere which is initially at 0°C. This problem is referenced from [9]. The analytical solution is given as,

$$u(0, t) = 100 - 200 \sum_{n=1}^{\infty} \frac{1}{z_n J_1(z_n)} e^{-(z_n \alpha / c)^2 t} \quad (4.3)$$



(a) Output in Paraview



(b) Comparison with analytical results

Figure 4.9: Representation of temperature profile of center

Mesh

```
[Mesh]
  file="circle.msh"
[]
```

Figure 4.10: Mesh block of the input file

Here instead of using the mesh generator we created mesh in gmsh and called it using file. Remember, the mesh file should be present in the same directory.

kernels

This is a transient problem so we need to add this term in addition to the previous kernel. Remember, MOOSE considers each differential term of the heat conduction equation as a separate kernel

```
[time_derivative]
  type = HeatConductionTimeDerivative
  variable = T
[]
[]
```

Figure 4.11: Time Derivative addition in kernels

Postprocessors

```
[VectorPostprocessors]
  [t_sampler]
    type = LineValueSampler
    variable = T
    start_point = '0 0 0'
    end_point = '0.1 0 0'
    num_points = 20
    sort_by = x
  []
[]
```

Figure 4.12: Vector Postprocessor block

This block has nothing to do with the solution part of the problem. After the problem is solved you can request MOOSE to display some parameters in the command window itself by using this block.

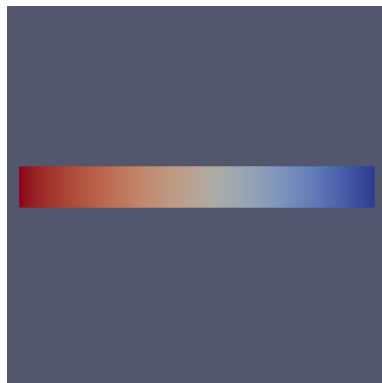
Transient heat conduction with variable thermal conductivity

This problem is referenced from an experiment carried out by [6]. Assume a K-Monel rod, 0.5 ft. long initially at 76°F. One end of the rod is at 32°F and the other end is at 600°F. The peripheral sides are insulated.

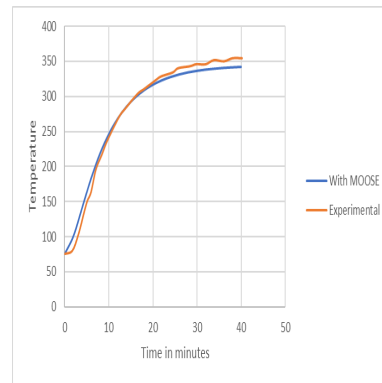
$$\rho = 523 \text{ lb}_m/\text{ft}^3 \quad (4.4)$$

$$c = 0.1075 \text{ Btu}/\text{lb}_m^\circ\text{F} \quad (4.5)$$

$$k = 9.5[1 + (0.00093/^\circ\text{F})t] \text{ Btu}/\text{hr ft}^\circ\text{F} \quad (4.6)$$



(a) Output in Paraview



(b) Comparison with experimental results

Figure 4.13: Representation of temperature profile of center

To adapt our kernel for a changing K , the information must be coded in the materials block of the input file as follows.

Material

```
[Materials]
[./property]
  type = GenericConstantMaterial
  prop_names = 'density specific_heat'
  prop_values = '523 0.1075'
[../]
[./thermal_conductivity]
  type = ParsedMaterial
  property_name = 'thermal_conductivity'
  coupled_variables = T
  expression = '9.5 * (1 + 0.00093* T)'
[../]
[]
```

Figure 4.14: Changes in material block

2D Dirichlet rectangle problem

This time we have a 2D problem where the top boundary of the rectangle is at 1 K and other boundaries are at 0 K. Thermal conductivity= 1. The analytic solution to this problem is given in reference [10].

$$\theta(x, y) = \frac{2}{\pi} \sum_{n=1}^{\infty} \frac{(-1)^{n+1} + 1}{n} \sin \frac{n\pi x}{L} \frac{\sinh n\pi y/L}{\sinh n\pi/L} \quad (4.7)$$

The above equation was plotted in Maple and results from both the software matched it.

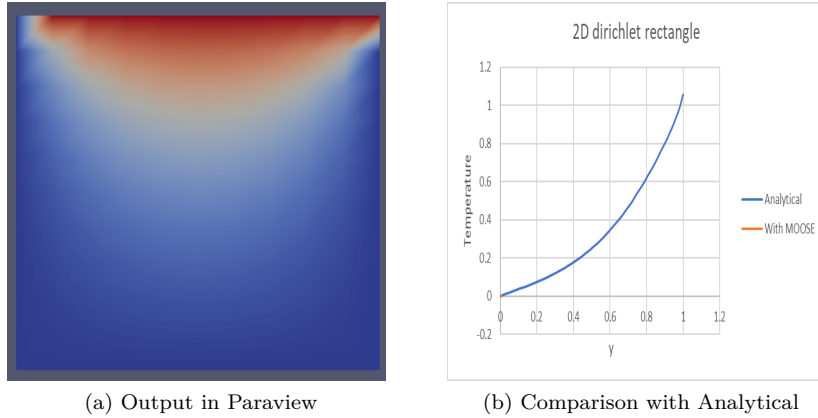


Figure 4.15: Representation of temperature profile

We also did this problem through our own kernel and got exact results.

A2: Application Development

In this section, we will present some examples of heat conduction for which we made the kernels or in other words formulated the weak form equation

1D heat conduction

We have considered a rectangle whose left end is at $T = 0$ K and the right end at $T = 100$ K. The other two ends are insulated so we have a 1-D problem at hand. As visible in the image, we solved this fundamental problem using 4 types of kernels and obtained exactly the same results in each case.

The kernels were also tested for volumetric heat problems and 2D Dirichlet rectangle problems and they returned identical results.

For using our own application we need to change the type to the kernel file created by us as shown below.

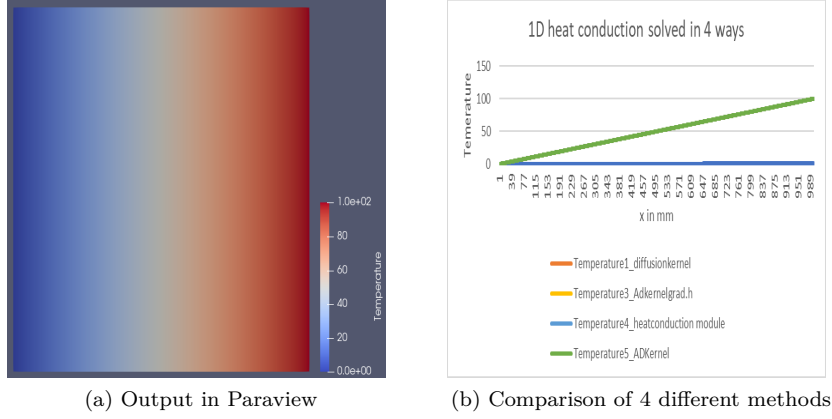


Figure 4.16: Representation of temperature profile along $y=0.5$

```
[kernels]
[temp4]
  type = Temptry1
  variable = Temperature
  thermal_conductivity = 1
[]
[]
```

Figure 4.17: Change in input file for using own kernel

To create the kernel we have to make two files, "filename.h" to be stored in Appname\include\kernels and "filename.C" to be stored in Appname\src\Kernels. In "filename.C" we need to provide the weak form that is required to solve the problem. We have presented the Temptry1.C file for understanding. Focus more on the second last line.

```
#include "Temptry1.h"

registerMooseObject("BabblerApp", Temptry1);

InputParameters
Temptry1::validParams()
{
  InputParameters params = ADKernel::validParams();
  params.addClassDescription("Compute the diffusion term for Temptry1 equation");
  params.addRequiredParam<Real>("thermal_conductivity", "The thermal_conductivity"
    "($K$) of the medium.");
  params.addParam<Real>("heatsource", 0, "The heat stored per unit volume"
    "of the medium.");
  return params;
}

Temptry1::Temptry1(const InputParameters& parameters)
: ADKernel(parameters),
  thermal_conductivity(getParam<Real>("thermal_conductivity")),
  _heatsource(getParam<Real>("heatsource"))
{
}

ADReal
Temptry1::computeQpResidual()
{
  return -_thermal_conductivity * (_grad_test[_i][_qp] * _grad_u[_qp]) + _heatsource * _test[_i][_qp];
}
```

Figure 4.18: Temptry1.C file in src-kernels

A3: Tensor Mechanics Module

Bi-axial loading

A 2D plane stress element is subjected to a biaxial state of stress, σ_{11} and $\sigma_{22} = 1$ respectively. $E = 1000$ and $\nu = 0.25$. From Hooke's law, we can find the strain components in both directions,

$$\epsilon_{11} = \frac{1}{E}(\sigma_{11} - \nu\sigma_{22}) = 0.75 \times 10^{-3}, \quad \epsilon_{22} = \frac{1}{E}(\sigma_{22} - \nu\sigma_{11}) = 0.75 \times 10^{-3} \quad (4.8)$$

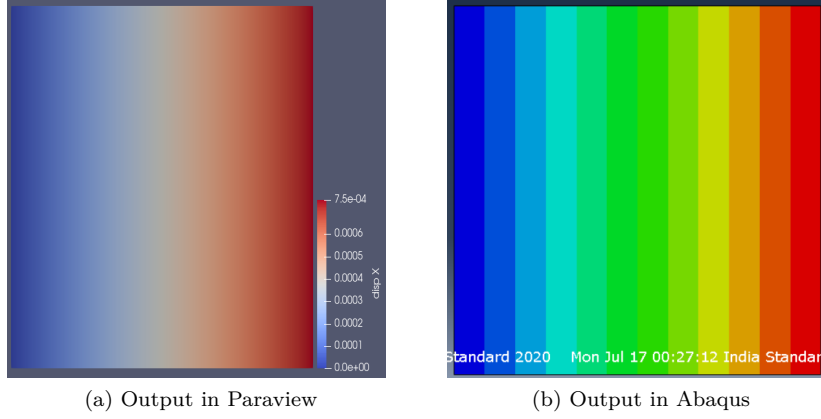


Figure 4.19: Representation of displacement profile along x-axis

We get the exact results in both Abaqus and MOOSE as expected. One point that is to be noted here is that if we take the value of E very less such as 1 for simulation in MOOSE, wrong results will be displayed. This may happen because of a large deformation occurring in the solid and we have set *strain=small and type=ComputeLinearElasticStress* in the input file. It is suggested to use a high value of E .

Hole in a plate subjected to uniaxial loading

For this problem, we have considered a plate that is 30 m in length and 20 m in breadth. $E = 1000$ and $\nu = 0.25$. Because of the symmetry of the problem, we are able to model only one-fourth of the plate.

The stress concentration factor at the tip of the hole in this case is equal to 3.

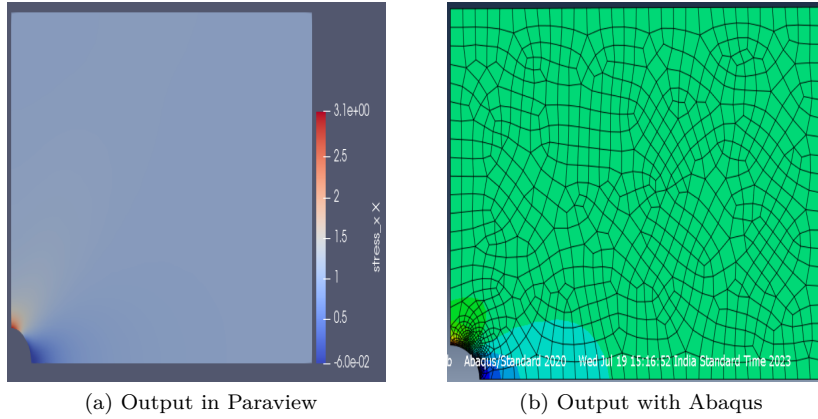


Figure 4.20: Representation of S11

This result is accurately matched by both software when we take a large number of elements near the periphery. In this BCs of the left and bottom face need to be taken care of, along with pressure.

Thick cylindrical pressure vessel

We have considered a thick cylinder having $r_i = 1$ and $r_o = 2$. $E = 1000$ and $\nu = 0.25$. Radial stress (σ_{rr}) in this case is given by,

$$\sigma_{rr} = \frac{P_i r_i^2 - P_o r_o^2}{r_o^2 - r_i^2} - \frac{P_i - P_o}{(r_o^2 - r_i^2)} r^2 \frac{r_i^2}{r_o^2} \quad (4.9)$$

$$\sigma_{\theta\theta} = \frac{P_i r_i^2 - P_0 r_0^2}{r_0^2 - r_i^2} + \frac{P_i - P_0}{(r_0^2 - r_i^2) r^2} r_i^2 r_0^2 \quad (4.10)$$

For $r = r_i$ $\sigma_{rr} = -1$ and $\sigma_{\theta\theta} = \frac{5}{3} = 1.667$ units respectively.

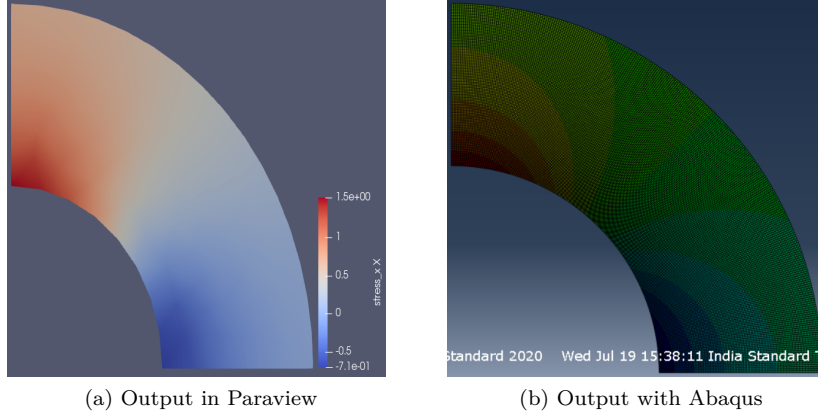


Figure 4.21: Representation of S11

Moose is showing 1.5 as the extreme value because we have used less number of nodes. Regarding code, we only need to add Pressure boundary condition = 1 at the inner surface.

A4: Input files for Phase-field benchmark problems

Uni-axial tension applied to a plain strain domain using Phase-field module

One can find crack2d_iso_wo_time.i [12] along the path given below:
\projects\moose\modules\combined\test\tests\phase_field_fracture
Our input file is based on this file only

```
#This input does not add time derivative kernel for phase field equation
[Mesh]
[gen]
  type = GeneratedMeshGenerator
  dim = 2
  nx = 1
  ny = 1
  ymax = 1
[]
[pin1]
  type = ExtraNodesetGenerator
  input = gen
  new_boundary = pin1
  coord = '0 0 0'
[]
[pin2]
  type = ExtraNodesetGenerator
  input = pin1
  new_boundary = pin2
  coord = '1 0 0'
[]
[pin3]
  type = ExtraNodesetGenerator
  input = pin2
  new_boundary = pin3
```

```

    coord = '0 1 0'
[]
[pin4]
    type = ExtraNodesetGenerator
    input = pin3
    new_boundary = pin4
    coord = '1 1 0'
[]
[]

[GlobalParams]
    displacements = 'disp_x disp_y'
[]

[Modules]
    [./TensorMechanics]
        [./Master]
            [./mech]
                add_variables = true
                strain = SMALL
                additional_generate_output = 'stress_yy'
                save_in = 'resid_x resid_y'
            [../]
        [../]
    [../]
[]

[Variables]
    [./c]
        order = FIRST
        family = LAGRANGE
    [../]
[]

[AuxVariables]
    [./resid_x]
    [../]
    [./resid_y]
    [../]
[]

[Kernels]
    [./solid_x]
        type = PhaseFieldFractureMechanicsOffDiag
        variable = disp_x
        component = 0
        c = c
    [../]
    [./solid_y]
        type = PhaseFieldFractureMechanicsOffDiag
        variable = disp_y
        component = 1
        c = c
    [../]
    [./ACBulk]
        type = AllenCahn
        variable = c
        f_name = F

```

```

[../]

[./ACInterface]
    type = ACInterface
    variable = c
    kappa_name = kappa_op
[../]
[]

[BCs]
[./ydisp]
    type = FunctionDirichletBC
    variable = disp_y
    boundary = top
    function = 't'
[../]
[./yfixpin1]
    type = DirichletBC
    variable = disp_y
    boundary = pin1
    value = 0
[../]
[./xfixpin1]
    type = DirichletBC
    variable = disp_x
    boundary = pin1
    value = 0
[../]
[./yfixpin2]
    type = DirichletBC
    variable = disp_y
    boundary = pin2
    value = 0
[../]
[./xfixpin2]
    type = DirichletBC
    variable = disp_x
    boundary = pin2
    value = 0
[../]
[./xfixpin3]
    type = DirichletBC
    variable = disp_x
    boundary = pin3
    value = 0
[../]
[./xfixpin4]
    type = DirichletBC
    variable = disp_x
    boundary = pin4
    value = 0
[../]
[]

[Materials]
[./pfbulkmat]
    type = GenericConstantMaterial
    prop_names = 'gc_prop 1 visco'

```

```

    prop_values = '5e-3 0.1 1e-8'
[../]
[/define_mobility]
    type = ParsedMaterial
    material_property_names = 'gc_prop visco'
    property_name = L
    expression = '1.0/(gc_prop * visco)'
[../]
[/define_kappa]
    type = ParsedMaterial
    material_property_names = 'gc_prop l'
    property_name = kappa_op
    expression = 'gc_prop * l'
[../]
[/elasticity_tensor]
    type = ComputeElasticityTensor
    C_ijkl = '121.15 80.77'
    fill_method = symmetric_isotropic
[../]
[/elastic]
    type = ComputeLinearElasticPFFractureStress
    c = c
    E_name = 'elastic_energy'
    D_name = 'degradation'
    F_name = 'local_fracture_energy'
    decomposition_type = strain_spectral
    use_current_history_variable = true
    use_snes_vi_solver = true
[../]
[/degradation]
    type = DerivativeParsedMaterial
    property_name = degradation
    coupled_variables = 'c'
    expression = '(1.0-c)^2*(1.0 - eta) + eta'
    constant_names = 'eta'
    constant_expressions = '0.0'
    derivative_order = 2
[../]
[/local_fracture_energy]
    type = DerivativeParsedMaterial
    property_name = local_fracture_energy
    coupled_variables = 'c'
    material_property_names = 'gc_prop l'
    expression = 'c^2 * gc_prop / 2 / l'
    derivative_order = 2
[../]
[/fracture_driving_energy]
    type = DerivativeSumMaterial
    coupled_variables = c
    sum_materials = 'elastic_energy local_fracture_energy'
    derivative_order = 2
    property_name = F
[../]
[]

[Postprocessors]
[/resid_x]
    type = NodalSum

```

```

        variable = resid_x
        boundary = 2
    [../]
    [./resid_y]
        type = NodalSum
        variable = resid_y
        boundary = 2
    [../]
[]

[Preconditioning]
    [./smp]
        type = SMP
        full = true
    [../]
[]

[Executioner]
    type = Transient

    solve_type = PJFNK
    petsc_options_iname = '-pc_type -snes_type'
    petsc_options_value = 'lu vinewtonrsls'

    nl_rel_tol = 1e-8
    l_max_its = 10
    nl_max_its = 10

    dt = 1e-3
    dtmin = 1e-3
    num_steps = 100
[]

[Outputs]
    exodus = true
[]

```

Al₂O₃ tension test using a sharp crack using staggered scheme

Code for this problem is based on tutorials given in [7] for Mode-I fracture

First, we have the solid mechanics file graded_tensiletest_Al2O3_usingstitch.i

```

        E = 3.8e5
    nu = 0.26
    K = '${fparse E/3/(1-2*nu)}'
    G = '${fparse E/2/(1+nu)}'

    Gc = 0.0663
    l = 0.003

[MultiApps]
    [fracture]
        type = TransientMultiApp
        input_files = fracture_tensiontest_graded_Al2O3_usingstitch.i
        cli_args = 'Gc=${Gc};l=${l}'
        execute_on = 'Timestep_END'
    []
[]

```

```

[Transfers]
  [from_d]
    type = MultiAppCopyTransfer
    multi_app = fracture
    direction = from_multiapp
    variable = d
    source_variable = d
  []
  [to_psie_active]
    type = MultiAppCopyTransfer
    multi_app = fracture
    direction = to_multiapp
    variable = psie_active
    source_variable = psie_active
  []
[]

[GlobalParams]
# order = FIRST
# family = LAGRANGE
displacements = 'disp_x disp_y'
# out_of_plane_strain = strain_zz
# pspg = true
[]

#
[Mesh]
  [top_half]
    type = FileMeshGenerator
    file = "graded_mesh_final_woc_tophalf.msh"
    # boundary_id_offset = 0
    # boundary_name_prefix = top_half
  []
  [top_stitch]
    type = BoundingBoxNodeSetGenerator
    input = top_half
    new_boundary = top_stitch
    bottom_left = '0.5 0 0'
    top_right = '1 0 0'
  []
  [bottom_half]
    type = FileMeshGenerator
    file = "graded_mesh_final_woc_bottomhalf.msh"
    # boundary_id_offset = 5
    # boundary_name_prefix = bottom_half
  []
  [bottom_stitch]
    type = BoundingBoxNodeSetGenerator
    input = bottom_half
    new_boundary = bottom_stitch
    bottom_left = '0.5 0 0'
    top_right = '1 0 0'
  []
  [stitch]
    type = StitchedMeshGenerator
    inputs = 'top_stitch bottom_stitch'
    stitch_boundaries_pairs = 'top_stitch bottom_stitch'
  []

```

```

    construct_side_list_from_node_list = true
[]

[Functions]
[./dts]
    type = PiecewiseConstant
    #x = '0 2.7e-3 5e-3'
    #y = '0 1e-3 1e-4'
    x = '0 8e-4 8.2e-4 8.4e-4 1e-3'
    y = '0 1e-4 1e-5 1e-6 1e-4'
direction = right
[./]
[]

[Variables]
[disp_x]
[]
[disp_y]
[]

# [./strain_zz]
# [./]

[]

[AuxVariables]
[fx]
[]
[fy]
[]
[d]
[]
[]

[]

[Physics/SolidMechanics/QuasiStatic]
[./plane_stress]
    # planar_formulation = WEAK_PLANE_STRESS
    add_variables = true
    strain = SMALL
    generate_output = 'stress_xx stress_xy stress_yy stress_zz strain_xx strain_xy strain_yy'
    #eigenstrain_names = eigenstrain
    use_automatic_differentiation = true
    save_in = 'fx fy'
[./]
[]

[BCs]
[top_y]
    type = ADFunctionDirichletBC
    variable = disp_y
    boundary = top
    function = 't'
[]
# [top_x]
# type = ADDirichletBC
# variable = disp_x
# boundary = top
# value = 0

```

```

# []
[bottom_y]
    type = ADDirichletBC
    variable = disp_y
    boundary = bottom
    value = 0
[]
[bottom_x]
    type = ADDirichletBC
    variable = disp_x
    boundary = origin
    value = 0
[]
# [left_x]
#     type = ADDirichletBC
#     variable = disp_x
#     boundary = left
#     value = 0
# []
[]

[Materials]
[bulk]
    type = ADGenericConstantMaterial
    prop_names = 'K G'
    prop_values = '${K} ${G}'
[]
[degradation]
    type = PowerDegradationFunction
    f_name = g
    function = (1-d)^p*(1-eta)+eta
    phase_field = d
    parameter_names = 'p eta '
    parameter_values = '2 1e-6'
[]
# [strain]
#     type = ADComputeSmallStrain
# []
[elasticity]
    type = SmallDeformationIsotropicElasticity
    bulk_modulus = K
    shear_modulus = G
    phase_field = d
    degradation_function = g
    decomposition = SPECTRAL
    output_properties = 'elastic_strain psie_active'
    outputs = exodus
[]
[stress]
    type = ComputeSmallDeformationStress
    elasticity_model = elasticity
    output_properties = 'stress'
    outputs = exodus
[]
[]

[Postprocessors]
[Fy]

```

```

        type = NodalSum
        variable = fy
        boundary = top
    []
    [./change_over_time1]
        type = ChangeOverFixedPointPostprocessor
        postprocessor = 'Fy'
        change_with_respect_to_initial = false
        execute_on = 'MULTIAPP_FIXED_POINT_END'
    [../]
[]

[Executioner]
    type = Transient

    solve_type = NEWTON
    petsc_options_iname = '-pc_type -pc_factor_mat_solver_package'
    petsc_options_value = 'lu          superlu_dist          ',
    automatic_scaling = true

    nl_rel_tol = 1e-6
    nl_abs_tol = 1e-8

    start_time = 0
    end_time = 1e-3

    fixed_point_max_its = 20
    fixed_point_min_its = 2
    accept_on_max_fixed_point_iteration = true
    fixed_point_rel_tol = 1e-6
    fixed_point_abs_tol = 1e-8

    [./TimeStepper]
        type = FunctionDT
        function = dts
    [../]
[]

[Outputs]
    exodus = true
    checkpoint = true
    print_linear_residuals = false
[]

```

Now, let's look at the fracture file, fracture_tensiontest_graded_Al2O3_usingstitch.i

```

[Mesh]
[top_half]
    type = FileMeshGenerator
    file = "graded_mesh_final_woc_tophalf.msh"
    # boundary_id_offset = 0
    # boundary_name_prefix = top_half
[]
[top_stitch]
    type = BoundingBoxNodeSetGenerator
    input = top_half
    new_boundary = top_stitch
    bottom_left = '0.5 0 0'

```

```

    top_right = '1 0 0'
[]
[bottom_half]
    type = FileMeshGenerator
    file = "graded_mesh_final_woc_bottomhalf.msh"
# boundary_id_offset = 5
# boundary_name_prefix = bottom_half
[]
[bottom_stitch]
    type = BoundingBoxNodeSetGenerator
    input = bottom_half
    new_boundary = bottom_stitch
    bottom_left = '0.5 0 0'
    top_right = '1 0 0'
[]
[stitch]
    type = StitchedMeshGenerator
    inputs = 'top_stitch bottom_stitch'
    stitch_boundaries_pairs = 'top_stitch bottom_stitch'
[]
construct_side_list_from_node_list = true
[]

[Variables]
[d]
[]
[]

[AuxVariables]
[bounds_dummy]
[]
[psie_active]
    order = CONSTANT
    family = MONOMIAL
[]
[]

[Bounds]
[irreversibility]
    type = VariableOldValueBoundsAux
    variable = bounds_dummy
    bounded_variable = d
    bound_type = lower
[]
[upper]
    type = ConstantBoundsAux
    variable = bounds_dummy
    bounded_variable = d
    bound_type = upper
    bound_value = 1
[]
[]

[Kernels]
[diff]
    type = ADPFFDiffusion
    variable = d
    fracture_toughness = Gc

```

```

        regularization_length = 1
        normalization_constant = c0
    []
[source]
    type = ADPFFSource
    variable = d
    free_energy = psi
[]
[]

[Materials]
[fracture_properties]
    type = ADGenericConstantMaterial
    prop_names = 'Gc l'
    prop_values = '${Gc} ${l}'
[]
[degradation]
    type = PowerDegradationFunction
    f_name = g
    function = (1-d)^p*(1-eta)+eta
    phase_field = d
    parameter_names = 'p eta '
    parameter_values = '2 1e-6'
[]
[crack_geometric]
    type = CrackGeometricFunction
    f_name = alpha
    function = 'd^2'
    phase_field = d
[]
[psi]
    type = ADDerivativeParsedMaterial
    f_name = psi
    function = 'alpha*Gc/c0/l+g*psie_active'
    args = 'd psie_active'
    material_property_names = 'alpha(d) g(d) Gc c0 l'
    derivative_order = 1
[]
[]

[Executioner]
    type = Transient

    solve_type = NEWTON
    petsc_options_iname = '-pc_type -pc_factor_mat_solver_package -snes_type'
    petsc_options_value = 'lu          superlu_dist          vinewtonrsls'
    automatic_scaling = true

    nl_rel_tol = 1e-6
    nl_abs_tol = 1e-8
[]

[Outputs]
    print_linear_residuals = false
[]

```

gmsh .geo file for initial crack representation through phase-field parameter

.geo file is created first, followed by .msh file that is used in the input file. Here, we are presenting .geo code for Miehe Tension test.

```
// Gmsh project created on Wed May 29 12:31:54 2024
//SetFactory("OpenCASCADE");
Geometry.CopyMeshingMethod =1;
//+
Point(1) = {0, 0.001, 0, 0.1};
//+
Point(2) = {0.27, 0.003, 0, 0.1};
//+
Point(3) = {0.47, 0.003, 0, 0.1};
//+
Point(4) = {0.49, 0.003, 0, 0.1};
//+
Point(5) = {0.5, 0.001, 0, 0.1};
//+
Point(6) = {1, 0.001, 0, 0.1};
//+
Point(7) = {1, 0.006, 0, 0.1};
//+
Point(8) = {1, 0.021, 0, 0.1};
//+
Point(9) = {1, 0.121, 0, 0.1};
//+
Point(10) = {0.5, 0.006, 0, 0.1};
//+
Point(11) = {0.5, 0.021, 0, 0.1};
//+
Point(12) = {0.5, 0.121, 0, 0.1};
//+
Point(13) = {1, 0.5, 0, 0.1};
//+
Point(14) = {0, 0.5, 0, 0.1};
//+
Point(15) = {0.07, 0.003, 0, 0.1};
//+
Point(16) = {1, 0.221, 0, 0.1};
//+
Point(17) = {0.5, 0.221, 0, 0.1};
//+
Point(18) = {0.5, 0.321, 0, 0.1};
//+
Point(19) = {1, 0.321, 0, 0.1};
//+
Point(20) = {0, 0.21, 0, 0.1};
//+
Point(21) = {0, 0, 0, 0.1};
//+
Point(22) = {1, 0, 0, 0.1};
//+
Point(24) = {0, 0.003, 0, 0.1};
//+
Point(25) = {0.5, 0.003, 0, 0.1};
//+
Point(26) = {0.5, 0, 0, 0.1};
```

```

//+
Line(1) = {1, 5};
//+
Ellipse(2) = {15, 25, 15, 17};
//+
Line(3) = {18, 19};
//+
Line(4) = {15, 2};
//+
Ellipse(5) = {2, 25, 2, 12};
//+
Line(6) = {17, 16};
//+
Line(7) = {2, 3};
//+
Ellipse(8) = {3, 25, 3, 11};
//+
Ellipse(9) = {4, 25, 4, 10};
//+
Ellipse(10) = {20, 5, 1, 18};
//+
Line(11) = {24, 20};
//+
Line(12) = {20, 14};
//+
Line(13) = {14, 13};
//+
Line(14) = {13, 19};
//+
Line(15) = {19, 16};
//+
Line(16) = {16, 9};
//+
Line(17) = {9, 8};
//+
Line(18) = {8, 7};
//+
Line(19) = {7, 6};
//+
Line(20) = {6, 22};
//+
Line(21) = {12, 9};
//+
Line(22) = {11, 8};
//+
Line(23) = {10, 7};
//+
Line(24) = {5, 6};
//+
Line(25) = {3, 4};
//+
Line(27) = {21, 26};
//+
Line(28) = {1, 21};
//+
Line(31) = {24, 15};
//+
Line(32) = {25, 5};

```

```

//+
Line(33) = {5, 26};
//+
Line(34) = {26, 22};
//+
Line(35) = {4, 25};
//+
Line(36) = {1, 24};
//+
Curve Loop(1) = {10, 3, -14, -13, -12};
//+
Plane Surface(1) = {1};
//+
Curve Loop(3) = {5, 21, -16, -6, -2, 4};
//+
Plane Surface(3) = {3};
//+
Curve Loop(4) = {22, -17, -21, -5, 7, 8};
//+
Plane Surface(4) = {4};
//+
Curve Loop(5) = {9, 23, -18, -22, -8, 25};
//+
Plane Surface(5) = {5};
//+
Curve Loop(8) = {-20, -24};
//+
Plane Surface(8) = {8};
//+
Curve Loop(9) = {31, 2, 6, -15, -3, -10, -11};
//+
Plane Surface(9) = {9};
//+
Curve Loop(10) = {1, -32, -35, -25, -7, -4, -31, -36};
//+
Plane Surface(10) = {10};
//+
Curve Loop(11) = {27, -33, -1, 28};
//+
Plane Surface(11) = {11};
//+
Curve Loop(12) = {34, -20, -24, 33};
//+
Plane Surface(12) = {12};
//+
Curve Loop(13) = {24, -19, -23, -9, 35, 32};
//+
Plane Surface(13) = {13};
//+
Transfinite Surface {1};
//+
Transfinite Surface {9};
//+
Transfinite Surface {3};
//+
Transfinite Surface {4};
//+
Transfinite Surface {5};

```

```

//+
Transfinite Surface {10};
//+
Transfinite Surface {11};
//+
Transfinite Surface {8};
//+
Transfinite Surface {12};
//+
Transfinite Surface {13};
//+
Transfinite Curve {24} = 841 Using Progression 1;
//+
Transfinite Curve {23} = 421 Using Progression 1;
//+
Transfinite Curve {9} = 21 Using Progression 1;
//+
Transfinite Curve {19} = 13 Using Progression 1;
//+
Transfinite Curve {25} = 4 Using Progression 1;
//+
Transfinite Curve {8} = 5 Using Progression 1;
//+
Transfinite Curve {22} = 26 Using Progression 1;
//+
Transfinite Curve {18} = 4 Using Progression 1;
//+
Transfinite Curve {7} = 11 Using Progression 1;
//+
Transfinite Curve {5} = 8 Using Progression 1;
//+
Transfinite Curve {21} = 11 Using Progression 1;
//+
Transfinite Curve {17} = 4 Using Progression 1;
//+
Transfinite Curve {4} = 11 Using Progression 1;
//+
Transfinite Curve {2} = 11 Using Progression 1;
//+
Transfinite Curve {6} = 9 Using Progression 1;
//+
Transfinite Curve {16} = 7 Using Progression 1;
//+
Transfinite Curve {1} = 51 Using Progression 1;
//+
Transfinite Curve {11} = 6 Using Progression 1;
//+
Transfinite Curve {10} = 6 Using Progression 1;
//+
Transfinite Curve {3} = 6 Using Progression 1;
//+
Transfinite Curve {15} = 4 Using Progression 1;
//+
Transfinite Curve {12} = 5 Using Progression 1;
//+
Transfinite Curve {13} = 10 Using Progression 1;
//+
Transfinite Curve {14} = 4 Using Progression 1;

```

```

//+
Transfinite Curve {28} = 3 Using Progression 1;
//+
Transfinite Curve {27} = 61 Using Progression 0.9;
//+
Transfinite Curve {20} = 6 Using Progression 1;
//+
Transfinite Curve {33} = 6 Using Progression 1;
//+
Transfinite Curve {32} = 2 Using Progression 1;
//+
Transfinite Curve {34} = 841 Using Progression 1;
//+
Transfinite Curve {35} = 16 Using Progression 1;
//+
Symmetry {0, 1, 0, 0} {
  Duplicata { Surface{1}; Surface{9}; Surface{3};Surface{4}; Surface{5}; Surface{10};Surface{11}
}
Coherence;
Mesh.SaveAll=1;
//+
Physical Curve("bottom", 97) = {41};
//+
Physical Curve("top", 98) = {13};
//+
Physical Point("origin", 99) = {42};

```

Changes in code for making a diffused pre-crack

For fracture file,

```

[Mesh]
  [fmg]
    type = FileMeshGenerator
    file = "miehe_mesh_final_wc2.msh"
  []
  [sideset]
    type = ParsedGenerateSideset
    input = fmg
    combinatorial_geometry = 'x<=0.5 & y<=0.001 & y>=-0.001'
    new_sideset_name = crack1
  []
[]
[BCs]
  [crack_bc]
    type = ADDirichletBC
    variable = d
    boundary = crack1
    value = 1
  []
[]

```

The same mesh is present for the elasticity file and d is declared as 1 for the initial timestep using Auxkernel.

Restart input file from a desired point

To utilize this feature checkpoint should be always set to true in the Output block of the input file.

```
[Mesh]
  file="graded_tensiletest_Zr02hypre_out_cp/0005-mesh.cpr"
[]
[Problem]
  restart_file_base = graded_tensiletest_Zr02hypre_out_cp/LATEST
[]
```

These two blocks will be there in both files. Also, the start time of the fracture file should be matched with the start time of the input file.

Bibliography

- [1] Marreddy Ambati, Tymofiy Gerasimov, and Laura De Lorenzis. A review on phase-field models of brittle fracture and a new fast hybrid formulation. *Computational Mechanics*, 55:383–405, 2015.
- [2] Hanen Amor, Jean-Jacques Marigo, and Corrado Maurini. Regularized formulation of the variational brittle fracture with unilateral contact: Numerical experiments. *Journal of the Mechanics and Physics of Solids*, 57(8):1209–1229, 2009.
- [3] Ted L Anderson. *Fracture mechanics: fundamentals and applications*. CRC press, 2017.
- [4] Tulio N Bittencourt, Paul A Wawrzynek, Anthony R Ingraffea, and JL Sousa. Quasi-automatic simulation of crack propagation for 2d lefm problems. *Engineering Fracture Mechanics*, 55(2):321–334, 1996.
- [5] Blaise Bourdin, Gilles A Francfort, and Jean-Jacques Marigo. Numerical experiments in revisited brittle fracture. *Journal of the Mechanics and Physics of Solids*, 48(4):797–826, 2000.
- [6] Earl Leonard Dowty. *Solutions to the transient heat conduction equation with variable thermal conductivity*. PhD thesis, Oklahoma State University, 1964.
- [7] Gary (Tianchen) Hu. Massively parallel fem code for phase-field for fracture by dolbow lab at duke university. <https://hugary1995.github.io/raccoon/>. RACCOON, 2022.
- [8] C. Geuzaine and J.-F. Remacle. Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, 79:1309–1331, 2009.
- [9] Michael D Greenberg. *Advanced Engineering Methods*. Prentice hall, 2001.
- [10] Frank P Incropera and David P DeWitt. Heat and mass transfer. *Fifth John Wiley and Sons*, pages 533–593, 2002.
- [11] Giovanni Lancioni and Gianni Royer-Carfagni. The variational approach to fracture mechanics. a practical application to the french panthéon in paris. *Journal of elasticity*, 95:1–30, 2009.
- [12] Alexander D. Lindsay, Derek R. Gaston, Cody J. Permann, Jason M. Miller, David Andrš, Andrew E. Slaughter, Fande Kong, Joshua Hansel, Robert W. Carlsen, Casey Icenhour, Logan Harbour, Guillaume L. Giudicelli, Roy H. Stogner, Peter German, Jacob Badger, Sudipta Biswas, Leora Chapuis, Christopher Green, Jason Hales, Tianchen Hu, Wen Jiang, Yeon Sang Jung, Christopher Matthews, Yinbin Miao, April Novak, John W. Peterson, Zachary M. Prince, Andrea Rovinelli, Sebastian Schunert, Daniel Schwen, Benjamin W. Spencer, Swetha Veeraraghavan, Antonio Recuero, Dewen Yushu, Yaqi Wang, Andy Wilkins, and Christopher Wong. 2.0 - MOOSE: Enabling massively parallel multiphysics simulation. *SoftwareX*, 20:101202, 2022.
- [13] Christian Miehe and Ercan Gürses. A robust algorithm for configurational-force-driven brittle crack propagation with r-adaptive mesh alignment. *International Journal for Numerical Methods in Engineering*, 72(2):127–155, 2007.

- [14] Christian Miehe, Martina Hofacker, and Fabian Welschinger. A phase field model for rate-independent crack propagation: Robust algorithmic implementation based on operator splits. *Computer Methods in Applied Mechanics and Engineering*, 199(45-48):2765–2778, 2010.
- [15] Christian Miehe, Fabian Welschinger, and Martina Hofacker. Thermodynamically consistent phase-field models of fracture: Variational principles and multi-field fe implementations. *International journal for numerical methods in engineering*, 83(10):1273–1311, 2010.
- [16] Sundararajan Natarajan, Ratna K Annabattula, Emilio Martínez-Pañeda, et al. Phase field modelling of crack propagation in functionally graded materials. *Composites Part B: Engineering*, 169:239–248, 2019.
- [17] N Singh, CV Verhoosel, R De Borst, and EH Van Brummelen. A fracture-controlled path-following technique for phase-field modeling of brittle fracture. *Finite Elements in Analysis and Design*, 113:14–29, 2016.
- [18] Vinut Raibagi. Phase field modelling of dynamic fracture and failure in fibre-reinforced composite laminate. M.Tech Thesis, IIT Bombay, 2022.
- [19] Jian-Ying Wu. A unified phase-field theory for the mechanics of damage and quasi-brittle failure. *Journal of the Mechanics and Physics of Solids*, 103:72–99, 2017.
- [20] Jian-Ying Wu and Vinh Phu Nguyen. A length scale insensitive phase-field damage model for brittle fracture. *Journal of the Mechanics and Physics of Solids*, 119:20–42, 2018.
- [21] Jian-Ying Wu, Vinh Phu Nguyen, Chi Thanh Nguyen, Danas Sutula, Sina Sinaie, and Stéphane PA Bordas. Phase-field modeling of fracture. *Advances in applied mechanics*, 53:1–183, 2020.
- [22] Shuwei Zhou, Timon Rabczuk, and Xiaoying Zhuang. Phase field modeling of quasi-static and dynamic crack propagation: Comsol implementation and case studies. *Advances in Engineering Software*, 122:31–49, 2018.